Assignment 1: Photoluminescence (PL) Spectroscopy Simulation

Course: Nanotechnology & Quantum Physics | **Topic:** instrument simulation, Kasha's rule, multi-dye spectroscopy

Simulator	Xenon arc lamp, excitation monochromator, harmonic filter, sample, detection monochromator, silicon photodiode
Sample database	Cy3, Cy5, methylene blue - absorption and emission bands digitized from vendor datasheets, PhotochemCAD and OMLC measurements
Physics	Beer-Lambert absorption, fluorescence quantum yield, Kasha's rule, monochromator passband convolution, detector responsivity, stray excitation light
Built with	Google Antigravity and a customized pi coding agent, using the DeepSeek v4.1 Flash and Gemini models
Source of numbers	every value quoted below is generated by the project code into *report/numbers.md*; every dye value is sourced in *data/SOURCES.md*

1. Design of the PL spectrometer and the three dye spectra (25%)

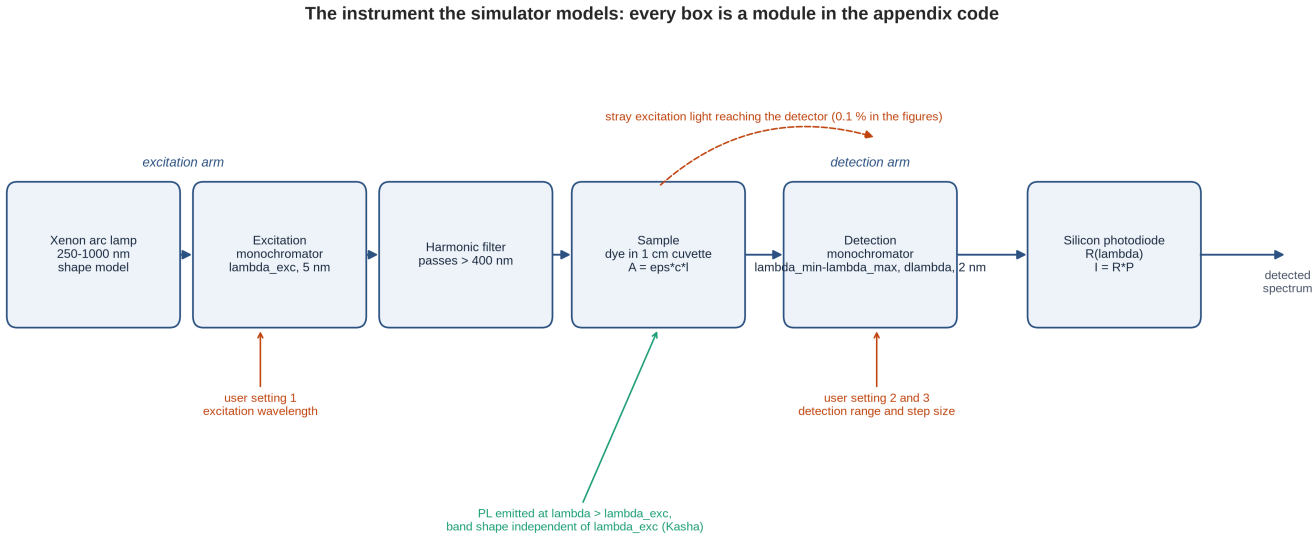


Figure 1: *the instrument the simulator models. Every box is a module in the appendix code: the lamp spectrum (sources.py), the monochromators and harmonic filter (sources.py), the dye database (samples.py, reference.py), the detector (detector.py) and the scan that connects them (spectrometer.py). The three user settings act where the drawing shows them. The instrument, the model and the analysis code were built with the AI tools named on the title page, working from the block diagram of the class session; every physical assumption, source datum and check is documented in the repository.*

1.1 The design.

A xenon arc lamp provides a broad continuum across the visible and near infrared. The excitation monochromator isolates the excitation wavelength λ_{exc} with a 5 nm passband; a long-pass harmonic filter removes the second-order grating output below 400 nm so the sample sees λ_{exc} and not $2\lambda_{exc}$. The beam crosses a 1 cm cuvette holding the dye. The emitted light is collected and scanned by a second monochromator, 2 nm passband, across the detection range [λ_{min}, λ_{max}] in steps of $\Delta\lambda$, and detected by a silicon photodiode whose responsivity $R(\lambda)$ grows through the visible and cuts off near 1100 nm. A small fraction of the excitation light reaches the detector as stray light; the figures that include it use 0.1 % of the excitation power, which is what a double monochromator with a harmonic filter is expected to leave. Intensities are relative: the model reproduces spectra and relative intensities, not absolute radiometric values.

What the sample does is described by two bands per dye, absorption and emission, taken from measured data rather than from ideal Gaussians. The absorbed fraction follows the Beer-Lambert law, so the emitted power is $P_{exc}(\lambda_{exc}) \times [1 - 10^{-A(\lambda_{exc})}] \times \Phi_f$, with $A = \varepsilon \cdot c \cdot l$. The emission band shape itself is independent of the excitation wavelength, which is the subject of section 2.

1.2 Sample database and baseline spectra.

Cy3 and Cy5 are the sulfoindocyanine dyes introduced as fluorescent labelling reagents, and their excitation ranges (546 nm for Cy3, 633-647 nm for Cy5) are the ones the model reproduces [3]. Methylene blue is a phenothiazinium dye whose excited state is quenched both by internal conversion and by intersystem crossing [4, 6], which is why its quantum yield is the lowest of the three. The database holds the three dyes named in the task, at the same concentration (1 µmol/L in a 1 cm cuvette, so $A = \varepsilon c l$ differs between them: 0.150 for Cy3, 0.250 for Cy5, 0.067 for methylene blue). Each dye was excited at its own absorption maximum. The spectra below are the instrument output, not idealised curves: they carry the lamp and monochromator throughput, the 2 nm detection passband and the detector responsivity. Their peaks fall where the measured bands peak - 568.0 nm for Cy3, 668.0 nm for Cy5 and 685.0 nm for methylene blue on the digitized curves, read as 686 nm on the 1 nm scan grid - which is a few nanometres away from the vendors' quoted maxima of 570, 670 and 685 nm. That difference is in the source data, not in the instrument: a symmetric passband cannot shift a band, it only widens it.

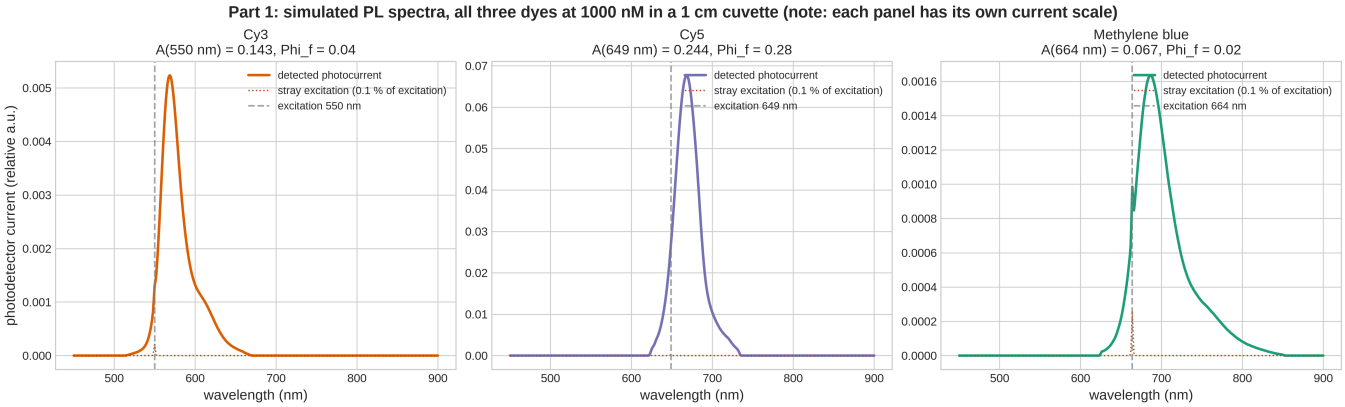


Figure 2: *simulated PL spectra of Cy3 (λ_{exc} = 550 nm), Cy5 (649 nm) and methylene blue (664 nm), all at 1 µmol/L in 1 cm, with the stray excitation line shown in each panel. Each panel has its own current scale; the peak absorbances differ by a factor of 3.7.*

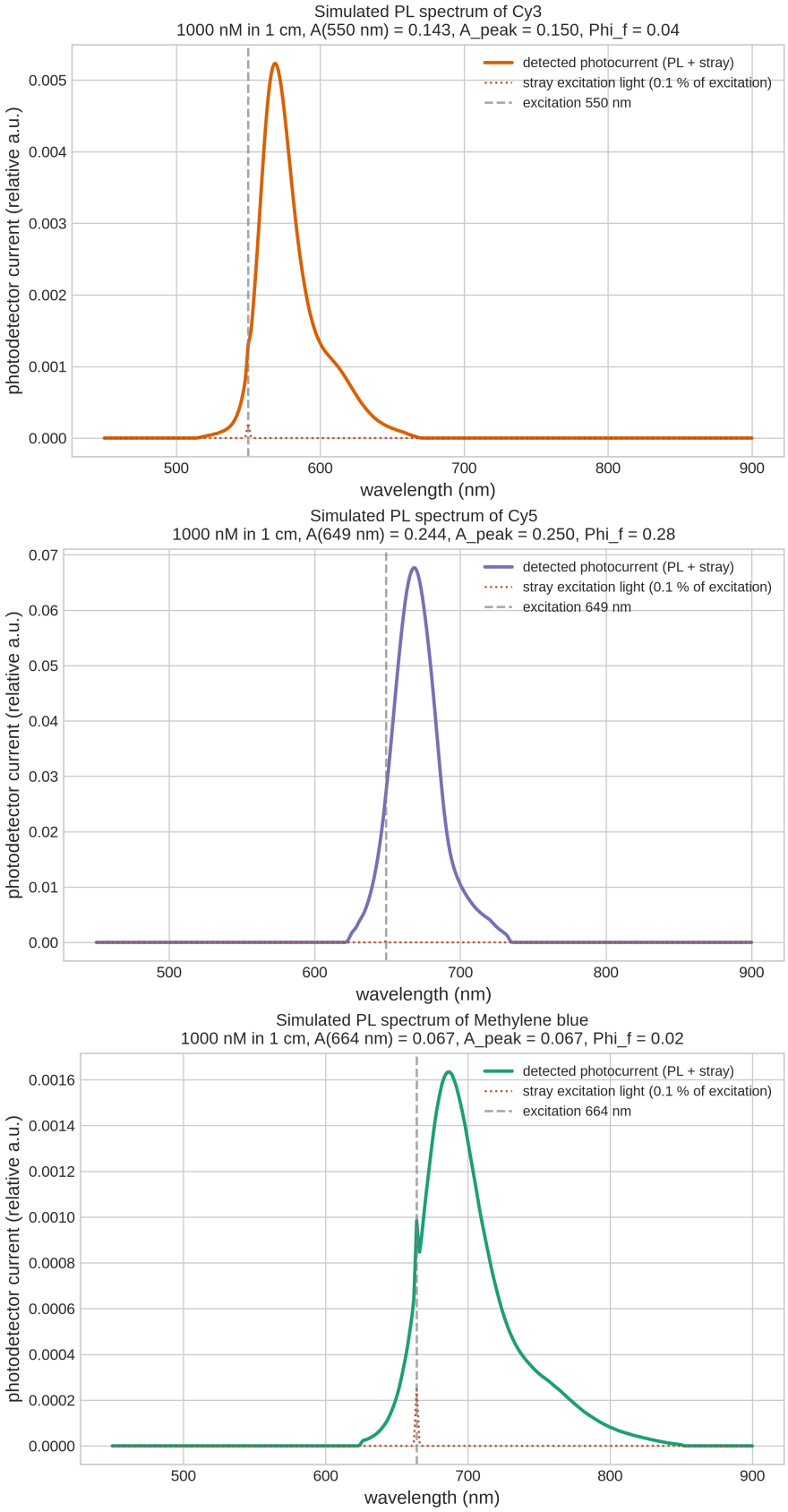


Figure 3: *the same three measurements at full scale, one per dye (these are the screen captures of the simulator output for question 1): A(λ_{exc}) and A_{peak} and Φ_f are printed in each title. Peak currents: Cy3 5.2 × 10^{-3}, Cy5 6.8 × 10^{-2}, methylene blue 1.6 × 10^{-3} (relative units).*

2. Excitation wavelength from 400 nm to 700 nm (50%)

2.1 What was done. Cy3 was chosen because its absorption (550 nm) and emission (570 nm) both sit comfortably inside the detector's useful range and it is the best documented of the three dyes. The excitation wavelength was swept from 400 nm to 700 nm in 2 nm steps while the whole emission band was recorded with the detection monochromator set to 450-900 nm in 1 nm steps, and the plotted intensity is the integral of the emitted band over the detection range, not the height at one fixed wavelength: a fixed window stops watching the dye and starts watching the window as soon as λ_{exc} moves past it.

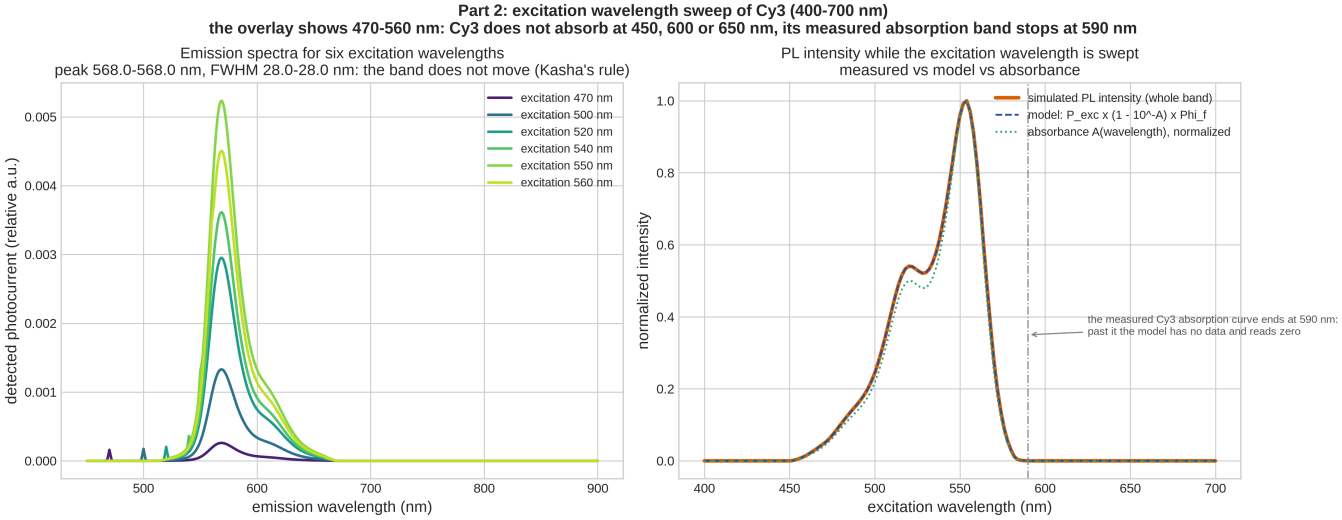


Figure 4: *(left) emission spectra for six excitation wavelengths, all peaking at 568.0 nm with 28.0 nm FWHM; (right) the integrated PL intensity across the sweep, the model prediction $P_{exc}(\lambda)[1 - 10^{-A(\lambda)}]\Phi_f$, and the absorbance normalised to its own maximum. The two model curves lie on top of each other; the absorbance differs slightly because the intensity carries the xenon lamp spectrum as a weight. The left panel is the detected signal including stray light, so each curve carries a small spike at its own excitation wavelength; the sweep on the right uses the fluorescence alone, so stray light cannot contaminate it.*

2.2 What is observed.

1. **The emission band does not move.** Across every excitation wavelength that produces a signal the peak stays at 568.0 nm and the FWHM at 28.0 nm on the 1 nm scan grid, 28.6 nm on the digitized band, and this is Kasha's rule in the data rather than in prose. The overlay in Figure 4 shows six spectra between 470 nm and 560 nm; the sweep itself is non-zero from about 452 nm to 588 nm.
2. **The height follows the absorbed light.** The measured integrated intensity equals $P_{exc}(\lambda)[1 - 10^{-A(\lambda)}]\Phi_f$ to within 10^{-16} of full scale, the numerical noise of the comparison. The curve rises towards the absorption maximum and collapses when the absorption does.
3. **Below about 450 nm the simulation has nothing to work with**: the digitized Cy3 absorption curve begins at 450 nm, so the model reads zero absorption below it. (The 400 nm harmonic filter does not enter here - it only blocks wavelengths below 400 nm, which the sweep does not use.) At 450 nm the dye genuinely absorbs very little; the pole of this limit is the tabulated range, not the physics.
4. **Above about 590 nm the simulation returns exactly zero.** This is a limit of the data, not of the physics: the digitized Cy3 absorption curve was measured from 450 nm to 590 nm, and the model reads zero outside that range. The measured methanol spectrum of the same chromophore (kept in *data/raw/*) shows the absorption tail continuing with a small but non-zero extinction coefficient at 650 nm, so a real instrument would record a weak, not vanishing, signal there.
5. **Stray light tracks the excitation wavelength.** With stray light enabled, the scattered line sits exactly at λ_{exc} and moves with it, which is why the harmonic filter and the second monochromator exist.

2.3 Do the observations fit the literature?

Yes, and the reason is a separation of timescales. Absorption promotes the molecule to higher vibrational levels of the first excited singlet state, or to higher electronic states. Internal conversion and vibrational relaxation then remove that excess energy in about 10^{-13}-10^{-12} s: Kasha 1950 puts an estimated upper bound of 10^{-13} s on the internal-conversion step from an intensity-ratio argument [1, p. 16], and that window is the modern textbook range for internal conversion and vibrational relaxation [2, §1.4.2], while fluorescence from Cy3 takes of order 0.2-2 ns [5]. Emission is therefore two to four orders of magnitude slower than the relaxation, so every excited molecule emits from the same thermalized level, $S_1(v = 0)$, whatever photon it absorbed. The emission spectrum is therefore fixed in shape, position and width, and only the number of excited molecules - that is, the absorbed light - changes with λ_{exc}. The two dyes in our own database illustrate the same chain-length argument: Cy3 is a trimethine cyanine absorbing at 550 nm and Cy5 a pentamethine at 649 nm, so two extra methine units shift the band by 99 nm in our own data, which is the free-electron model of the cyanine chain in practice [3, 7]. That is Kasha's rule as Kasha stated it for states [1] and as Lakowicz states it for wavelength, in the section whose title says exactly that [2, §1.3.2].

The intensity behaviour is the excitation spectrum, with one honest caveat. The absorbed photon rate follows Beer-Lambert, so $I_{PL} \propto I_0(\lambda_{exc})[1 - 10^{-\varepsilon(\lambda_{exc})cl}]\Phi_f$, which for a dilute sample is $\propto \varepsilon(\lambda_{exc})\Phi_f$. The curve in Figure 4 (right) therefore tracks the absorption band, but the lamp spectrum $I_0(\lambda)$ sits inside that expression, so the two curves are similar rather than identical - the residual difference visible in the figure is the xenon lamp, not an error. Correcting for $I_0(\lambda)$ is what an instrument does when it records a quantum-corrected excitation spectrum.

A note on what this simulation can and cannot demonstrate. The invariance of the band with λ_{exc} is built into the model, because that is what the timescale argument above requires. What the sweep verifies is the consequence: that the absorbed fraction, and nothing else, depends on the excitation wavelength, and that the resulting intensity agrees with the Beer-Lambert prediction to machine precision. In that sense the simulator is a test of the consequences of Kasha's rule rather than a discovery of it.

The Stokes shift. Absorption and emission maxima are separated by 20 nm, which is 638 cm^{-1} calculated from the quoted 550 and 570 nm values (the digitized curve peaks, 554 and 568 nm, give 14 nm and 445 cm^{-1}; the figure is quoted from the sourced maxima). The shift is the Franck-Condon and solvent reorganisation energy, and it is why the emission sits at longer wavelength than the absorption, never at shorter.

3. The three dyes on one graph at 450, 550 and 650 nm (25%)

3.1 The colours come from the absorption bands. All three dyes were simulated at the same concentration, 1 µmol/L in a 1 cm cuvette, so their peak absorbances differ by their extinction coefficients: 0.150 (Cy3), 0.250 (Cy5) and 0.067 (methylene blue). "The same concentration" is not "the same absorbance" - that is the first thing to say about this question. The second is that each dye sits at a different point of its own absorption curve, shown in Figure 5.

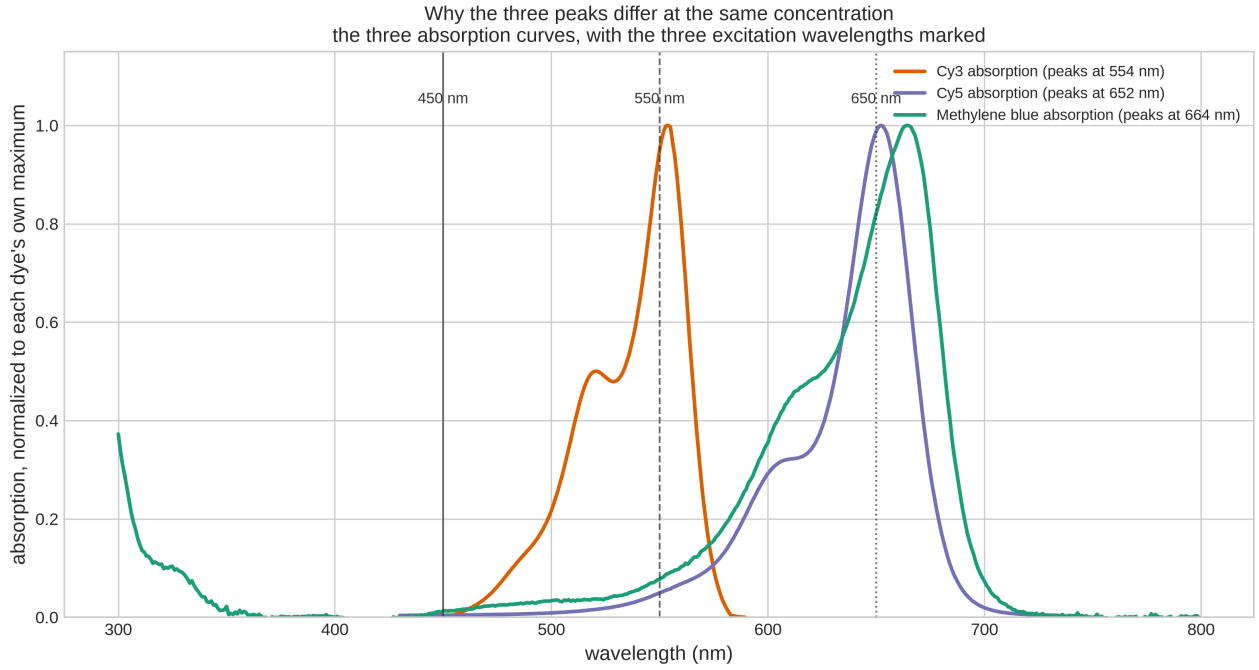


Figure 5: *the three absorption bands, each normalised to its own maximum, with the three excitation wavelengths marked. At 550 nm Cy3 is near its maximum (its curve peaks at 554 nm) while Cy5 and methylene blue are on their rising edges; at 650 nm the situation is reversed.*

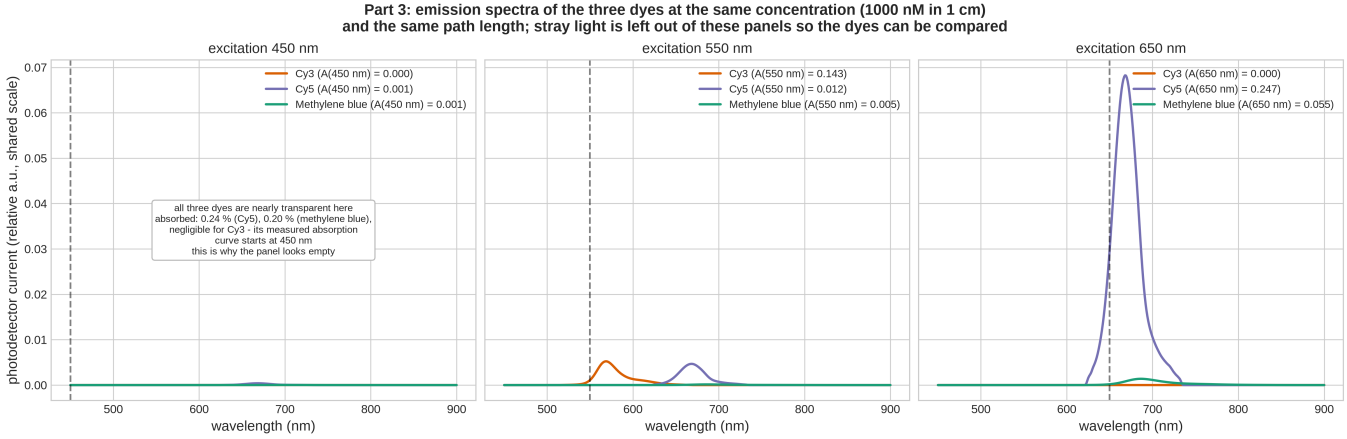


Figure 6: *the same graph for each excitation wavelength - 450 nm (left), 550 nm (centre), 650 nm (right) - on one shared scale in relative units, which is what makes the comparison meaningful. The legend gives $A(\lambda_{exc})$ for each curve in that panel.*

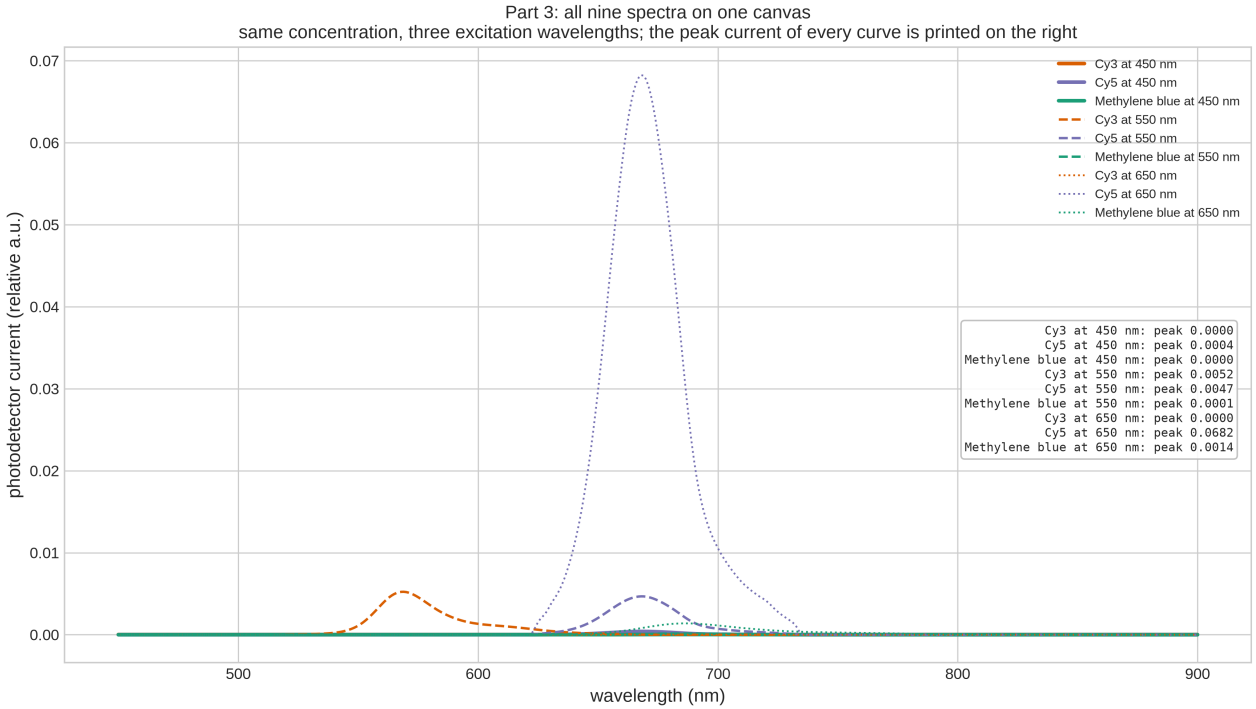


Figure 7: *all nine spectra on a single canvas, with the peak current of every curve printed beside it. The empty-looking curves are not an error: they are the dyes that do not absorb at that wavelength.*

3.2 Why the peaks are not the same.

The peak height is a product of four terms, and the first is the one the question is looking for:

(1) How much light the dye absorbs at λ_{exc}. $A(\lambda_{exc}) = \varepsilon(\lambda_{exc})cl$, and ε at a given wavelength depends entirely on where that wavelength falls in the dye's own band. At 550 nm Cy3 absorbs 28.0 % of the incident light while Cy5 absorbs 2.81 % and methylene blue 1.19 %. At 650 nm it is Cy5 that absorbs 43.4 %, methylene blue 11.9 %, and Cy3 nothing at all.
(2) The quantum yield. Of the photons absorbed, only the fraction Φ_f is emitted: 0.28 for Cy5, 0.04 for Cy3, 0.02 for methylene blue in water. This is why the 550 nm panel is not lopsided in favour of Cy3: Cy3 absorbs ten times more light than Cy5 there, yet the two peaks come out within 12 % of each other, because Cy5 converts absorbed photons into fluorescence seven times more efficiently.
(3) The detector's wavelength dependence. The silicon responsivity is about a third higher at methylene blue's emission (686 nm) than at Cy3's (568 nm), so equal emitted power at those two wavelengths does not give equal photocurrent.
(4) The width of the emission band. Measured on the digitized bands, Cy3, Cy5 and methylene blue are 28.6, 33.4 and 48.6 nm wide (the simulated curves read 28, 32 and 47 nm on a 1 nm grid). For the same emitted power, a broader band means a lower peak, which penalises methylene blue a second time on top of its low quantum yield.

One honest caveat about 450 nm. At that wavelength none of the three dyes absorbs meaningfully (0.24 % for Cy5, 0.20 % for methylene blue, and nothing measurable for Cy3), so the left panel of Figure 6 is nearly empty for a physical reason and not because the simulation failed. The comparison the question is really asking about is the 550 nm and 650 nm panels, where the dyes that absorb are also the dyes that light up.

Peak currents for the record (relative units, 1 µmol/L, 1 cm): at 450 nm Cy3 0.00000, Cy5 0.00040, methylene blue 0.00002; at 550 nm Cy3 0.00523, Cy5 0.00468, methylene blue 0.00015; at 650 nm Cy3 0.00000, Cy5 0.06824, methylene blue 0.00138.

4. References

Every entry below was checked to resolve, and each claim is attached to the work that carries it. Where a DOI circulates in a wrong form, the corrected one is given. Where a work is closed-access and could not be read beyond its table of contents, the citation points to the section whose stated subject carries the claim, and the claim is phrased as textbook content rather than as a quotation from a page that was not read.

[1] M. Kasha, "Characterization of electronic transitions in complex molecules", *Discussions of the Faraday Society* **9** (1950) 14-19. DOI 10.1039/DF9500900014. (Statement of the rule for states, pp. 14-15; internal conversion faster than emission, p. 16.)

[2] J. R. Lakowicz, *Principles of Fluorescence Spectroscopy*, 3rd ed., Springer, 2006. DOI 10.1007/978-0-387-46312-4. (§1.3.2, "Emission Spectra Are Typically Independent of the Excitation Wavelength", p. 7; §1.4.2, "Timescale of Molecular Processes in Solution", p. 12.)

[3] R. B. Mujumdar, L. A. Ernst, S. R. Mujumdar, C. J. Lewis and A. S. Waggoner, "Cyanine dye labeling reagents: sulfoindocyanine succinimidyl esters", *Bioconjugate Chemistry* **4** (1993) 105-111. DOI 10.1021/bc00020a001. (The Cy3/Cy5 chromophores and their excitation ranges.)

[4] E. M. Tuite and J. M. Kelly, "Photochemical interactions of methylene blue and analogues with DNA and other biological substrates", *Journal of Photochemistry and Photobiology B: Biology* **21** (1993) 103-124. DOI 10.1016/1011-1344(93)80173-7. (Excited-state behaviour of methylene blue.)

[5] R. Berrocal-Martin et al., *Journal of Physical Chemistry B* **129** (2025) 11626-11635. DOI 10.1021/acs.jpcb.5c03845. (Fluorescence lifetimes of Cy3 in solution: about 0.2 ns free dye, components of 0.6-1.1 ns and 1.5-2.5 ns.)

[6] L. M. Moreira et al., "Phenothiazinium dyes as photosensitizers in photodynamic therapy", InTech, 2012, p. 395. DOI 10.5772/48087. (Methylene blue: internal conversion and triplet formation, which is why its quantum yield is the lowest of the three.)

[7] H. Kuhn, "A quantum-mechanical theory of light absorption of organic dyes and similar compounds", *Journal of Chemical Physics* **17** (1949) 1198-1212. DOI 10.1063/1.1747143. (Free-electron model of the cyanine chain, eq. (8); the report uses it for the chain-length argument and quotes the shift from our own data - 550 nm for the trimethine Cy3 against 649 nm for the pentamethine Cy5.)

[8] Hamamatsu Photonics, "Si photodiodes", technical note KSPD9001E04, sec. 2-3, eq. (2-3), which prints the quantum-efficiency form QE = (S × 1240 / λ) × 100 %; equivalently the responsivity of a detector of quantum efficiency η is ηλ[µm]/1.24 A/W.

[9] Dye spectra and molar extinction coefficients: the Cytiva/GE Healthcare Cy3 amine-reactive sheet 28951573 (catalogue PA23001: 550 nm, 150 000, 570 nm) and the Cy5 maleimide sheet 28954683 (PA25031: 649 nm, 250 000, 670 nm; the amine-reactive code is PA25001); for methylene blue the PhotochemCAD record PSW009, whose own reference is JACS Au (DOI 10.1021/jacsau.5c01463), and the OMLC/Prahl measured spectrum. The full source table, with a confidence for each value and the raw files, is in *data/SOURCES.md* and *data/raw/*.

Declared limitations. No primary source was found for the widely repeated methylene blue extinction coefficient of 95,000 $M^{-1}cm^{-1}$, so it is not used: the pack carries 67,000 (PhotochemCAD, water), 74,028 (OMLC) and 107,800 for the deconvoluted monomer. The quantum yield of methylene blue in water, 0.02, is the value used in photodynamic-therapy work; the readable measurement in the literature is 0.04 in ethanol. The emission bandwidths quoted here are measured on the digitized bands rather than taken from the literature; a published Cy5 emission FWHM of 37.4 nm exists (Biosensors 2023, 13, 876, Table 2) against our 33.4 nm, and for Cy3 no published value was found at all. The FPbase records from which the Cy3 and Cy5 bands are digitized do not state their provenance; the fully documented curves for the same chromophores are the PhotochemCAD methanol spectra kept in *data/raw/*.

Appendix: complete source code

The files below are read from the repository at build time, so the appendix cannot drift from the code that produced the figures. Every result in this report is reproducible with *uv sync* followed by the two generator scripts listed here.

File: plsim/bands.py - *Band shapes: Gaussian model and tabulated measured bands*

```
"""Spectral band shapes."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np

FWHM_TO_SIGMA = 1.0 / 2.3548200450309493


def gaussian(wavelength_nm, center_nm, fwhm_nm, peak=1.0):
    """Gaussian band centred on center_nm, with the given FWHM and peak value."""
    wl = np.asarray(wavelength_nm, dtype=float)
    sigma = fwhm_nm * FWHM_TO_SIGMA
    return peak * np.exp(-0.5 * ((wl - center_nm) / sigma) ** 2)


@dataclass(frozen=True)
class GaussianBand:
    """A band with a Gaussian shape, described by centre and FWHM in nm."""

    center_nm: float
    fwhm_nm: float
    peak: float = 1.0

    def __call__(self, wavelength_nm):
        return gaussian(wavelength_nm, self.center_nm, self.fwhm_nm, self.peak)


@dataclass(frozen=True)
class TabulatedBand:
    """A measured band: points on a wavelength grid, interpolated in between.

    Outside the tabulated wavelength range the band is zero, so a spectrum from
    measured data is only defined where it was measured.
    """

    wavelength_nm: tuple
    value: tuple
    name: str = ""

    @classmethod
    def from_lists(cls, wavelength_nm, value, name=""):
        wavelengths = np.asarray(wavelength_nm, dtype=float)
        values = np.asarray(value, dtype=float)
        if wavelengths.size < 2 or wavelengths.size != values.size:
            raise ValueError("a tabulated band needs at least two matching wavelength/value points")
        return cls(tuple(wavelengths), tuple(values), name)

    def __call__(self, wavelength_nm):
        wavelengths = np.asarray(self.wavelength_nm, dtype=float)
        values = np.asarray(self.value, dtype=float)
        return np.interp(np.asarray(wavelength_nm, dtype=float), wavelengths, values, left=0.0, right=0.0)
```

File: plsim/sources.py - *Excitation arm: xenon lamp, monochromator, harmonic filter*

```
"""Excitation end: xenon lamp, monochromator, harmonic filter."""

from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np

from .bands import FWHM_TO_SIGMA, gaussian

# Relative amplitudes and wavelengths of the strongest xenon emission lines in
# the visible / near-infrared region. Illustrative values, not measured lamp data.
XE_LINES_NM = (
    (450.1, 0.20),
    (462.4, 0.32),
    (467.1, 0.40),
    (823.2, 0.55),
    (828.0, 0.35),
    (881.9, 0.50),
)

LAMP_GRID_NM = (200.0, 1100.0, 0.5)


@dataclass(frozen=True)
class XenonLamp:
    """Xenon arc lamp at the excitation end.

    The spectrum is a smooth approximation of the arc continuum (weak in the
    deep UV, rising through the blue, peaking in the green, tailing into the
    near infrared) plus the strongest xenon emission lines. It is a shape
    model, not a measured lamp spectrum.
    """

    continuum_peak_nm: float = 550.0
    continuum_fwhm_nm: float = 700.0
    line_strength: float = 0.5
    line_fwhm_nm: float = 2.5

    def spectral_radiance(self, wavelength_nm):
        """Relative spectral radiance."""
        continuum = gaussian(wavelength_nm, self.continuum_peak_nm, self.continuum_fwhm_nm)
        lines = np.zeros_like(np.asarray(wavelength_nm, dtype=float))
        for center, amplitude in XE_LINES_NM:
            lines += gaussian(wavelength_nm, center, self.line_fwhm_nm, amplitude)
        return continuum + self.line_strength * lines


@dataclass(frozen=True)
class Monochromator:
    """Grating monochromator: splits light and passes one wavelength band."""

    bandwidth_nm: float = 5.0
    throughput: float = 0.5
```

```
    def transmission(self, wavelength_nm, center_nm):
        return gaussian(wavelength_nm, center_nm, self.bandwidth_nm, self.throughput)

    def broaden(self, power, step_nm):
        """Pass a spectrum through the instrument: convolve it with the passband.

        A spectrum measured through the monochromator is the true spectrum
        smeared by the passband, so narrow features (like a stray excitation
        line) come out with the passband width.
        """
        sigma_nm = self.bandwidth_nm * FWHM_TO_SIGMA
        half_width_nm = 4.0 * sigma_nm
        offsets_nm = np.arange(-half_width_nm, half_width_nm + 0.5 * step_nm, step_nm)
        kernel = np.exp(-0.5 * (offsets_nm / sigma_nm) ** 2)
        kernel = kernel / (kernel.sum() * step_nm)
        return np.convolve(power, kernel, mode="same") * step_nm

    def line_at(self, wavelength_nm, center_nm, power):
        """Power density of a line of total power `power` centred on center_nm."""
        sigma_nm = self.bandwidth_nm * FWHM_TO_SIGMA
        return power * gaussian(wavelength_nm, center_nm, self.bandwidth_nm) / (sigma_nm * np.sqrt(2.0 * np.pi
    ))


@dataclass(frozen=True)
class HarmonicFilter:
    """Long-pass filter that blocks the harmonics leaving the monochromator.

    With the grating set to lambda, second-order diffraction also passes
    lambda/2. The filter removes that short-wavelength light before it reaches
    the sample, so the sample is excited by lambda_exc only.
    """

    cutoff_nm: float = 400.0
    blocked_transmission: float = 1e-3

    def transmission(self, wavelength_nm):
        wl = np.asarray(wavelength_nm, dtype=float)
        return np.where(wl >= self.cutoff_nm, 1.0, self.blocked_transmission)


@dataclass(frozen=True)
class ExcitationArm:
    """Lamp -> monochromator -> harmonic filter, delivering light to the sample."""

    lamp: XenonLamp = field(default_factory=XenonLamp)
    monochromator: Monochromator = field(default_factory=Monochromator)
    harmonic_filter: HarmonicFilter = field(default_factory=HarmonicFilter)

    def transmission_profile(self, wavelength_nm, lambda_exc_nm):
        """Fraction of the lamp light at each wavelength that reaches the sample."""
        return self.monochromator.transmission(
            wavelength_nm, lambda_exc_nm
        ) * self.harmonic_filter.transmission(wavelength_nm)
    def power_at(self, lambda_exc_nm):
        """Excitation power delivered to the sample, in relative units."""
        lo, hi, step = LAMP_GRID_NM
        grid = np.arange(lo, hi + step, step)
        profile = self.transmission_profile(grid, lambda_exc_nm)
        return float(np.trapezoid(self.lamp.spectral_radiance(grid) * profile, grid))
```

File: plsim/samples.py - *Sample database, absorbance, concentration*

```
"""Sample database: the dyes of the class task, as a Gaussian model.

Band centres and quantum yields are the sourced values from `data/SOURCES.md`,
where each number carries its solvent and its spread. The band shapes are
Gaussian approximations; `plsim/reference.py` loads the measured band shapes
for the same three dyes.
"""

from __future__ import annotations

import dataclasses
from dataclasses import dataclass

import numpy as np

from .bands import GaussianBand


@dataclass(frozen=True)
class Dye:
    """A fluorescent sample: one absorption band and one emission band."""

    name: str
    absorption: object
    emission: object
    quantum_yield: float
    peak_absorbance: float = 1.0
    solvent: str = "water"
    extinction_coefficient: float | None = None
    note: str = ""
    source: str = ""

    def absorbance(self, wavelength_nm):
        """Absorbance (optical density) of the sample at that wavelength."""
        return self.peak_absorbance * self.absorption(wavelength_nm)

    def absorbed_fraction(self, lambda_exc_nm):
        """Fraction of the excitation light the sample absorbs (A at the peak)."""
        return float(1.0 - 10.0 ** (-self.absorbance(lambda_exc_nm)))

    def emission_shape(self, wavelength_nm):
        """Emission band shape, 1 at its maximum."""
        return self.emission(wavelength_nm)

    def emitted_power(self, wavelength_nm, lambda_exc_nm, excitation_power):
        """PL power emitted per wavelength, in relative units.

        The emission band shape does not depend on the excitation wavelength.
        A molecule that has absorbed a photon relaxes to the bottom of S1 before
        it emits, so every excitation wavelength produces the same spectrum and
        only the amount of absorbed light changes with it (Kasha's rule, which
        is the subject of the assignment's second question).

        Two deliberate omissions, both documented here because they are the
        difference between a simulator that shows Kasha's rule and one that
```

```
        quietly breaks it:

        - No cut of the emission at lambda_exc. An earlier version multiplied the
          band by a step at lambda_exc, which truncated the band whenever the
          excitation fell inside it and made the simulated spectrum move with
          the excitation wavelength.
        - No anti-Stokes term. Anti-Stokes emission (emitted photon more energetic
          than the absorbed one) is weak, and adding a Boltzmann factor on top of
          a measured band double-counts the vibronic envelope that is already in
          the measured band shape.
        """
        return (
            excitation_power
            * self.absorbed_fraction(lambda_exc_nm)
            * self.quantum_yield
            * self.emission_shape(wavelength_nm)
        )


SAMPLE_DATABASE = {
    "cy3": Dye(
        name="Cy3",
        absorption=GaussianBand(center_nm=550.0, fwhm_nm=45.0),
        emission=GaussianBand(center_nm=570.0, fwhm_nm=28.6),
        quantum_yield=0.04,
        solvent="PBS, pH 7.4 (free dye)",
        extinction_coefficient=150000.0,
        note=(
            "Absorption 550 nm / emission 570 nm (vendor values; the literature spreads 544-555 nm and "
            "563-570 nm). Quantum yield 0.04 holds for the free dye in PBS; conjugates and sulfo-Cy3 are "
            "reported at 0.10-0.31, so quote it together with the solvent."
        ),
        source="data/SOURCES.md",
    ),
    "cy5": Dye(
        name="Cy5",
        absorption=GaussianBand(center_nm=649.0, fwhm_nm=37.7),
        emission=GaussianBand(center_nm=670.0, fwhm_nm=33.5),
        quantum_yield=0.28,
        solvent="water / PBS (free dye)",
        extinction_coefficient=250000.0,
        note=(
            "Absorption 649 nm / emission 670 nm. Quantum yield 0.28 is the value cited for Cy5 in aqueous "
            "solution; individual measurements span 0.19-0.30, and 0.4 in methanol is disputed by an "
            "integrating-sphere measurement of 0.17."
        ),
        source="data/SOURCES.md",
    ),
    "methylene-blue": Dye(
        name="Methylene blue",
        absorption=GaussianBand(center_nm=664.0, fwhm_nm=55.9),
        emission=GaussianBand(center_nm=685.0, fwhm_nm=48.7),
        quantum_yield=0.02,
        solvent="water",
        extinction_coefficient=67000.0,
```

```
        note=(
            "Absorption 664 nm / emission 685 nm. Quantum yield 0.02 in water, which is the value used when "
            "methylene blue serves as a reference in photodynamic therapy; the classic ethanol value is 0.04.
    "
            "The molar extinction coefficient depends on how the dimer is treated: 67,000-74,000 in dilute "
            "water as measured, 107,800 for the deconvoluted monomer."
        ),
        source="data/SOURCES.md",
    ),
}


def with_concentration(dye, concentration_M, path_length_cm=1.0):
    """The same dye at a given concentration, with the absorbance that follows.

    A_peak = epsilon * c * l, so the same concentration of two dyes does NOT mean
    the same peak absorbance unless their extinction coefficients match. Pass the
    result to the spectrometer when the experiment is "equal concentrations".
    """
    if dye.extinction_coefficient is None:
        raise ValueError("{} has no extinction coefficient, so no concentration can be applied".format(dye.nam
    e))
    return dataclasses.replace(
        dye, peak_absorbance=dye.extinction_coefficient * concentration_M * path_length_cm
    )


def normalise_key(name):
    """Dye names are matched ignoring case, spaces and dashes."""
    key = name.strip().lower().replace("_", "-").replace(" ", "-")
    return "methylene-blue" if key == "mb" else key


def get_sample(name, database=None):
    """Look a dye up by name in the given database (the Gaussian model by default)."""
    database = SAMPLE_DATABASE if database is None else database
    key = normalise_key(name)
    try:
        return database[key]
    except KeyError:
        raise KeyError(
            "unknown sample {!r}; available: {}".format(name, ", ".join(database))
        ) from None
```

File: plsim/reference.py - *Loader for the measured reference band data*

```
"""Measured reference data: digitized absorption and emission bands.

The pack in `data/reference_spectra.json` was collected from vendor spec
sheets, spectral databases and peer-reviewed papers; `data/SOURCES.md` gives a
source and a confidence for every number.
"""

from __future__ import annotations

import json
from pathlib import Path

from .bands import TabulatedBand
from .samples import Dye

DEFAULT_PATH = Path(__file__).resolve().parent.parent / "data" / "reference_spectra.json"
SOURCE = "data/reference_spectra.json (see data/SOURCES.md)"


def available(path=DEFAULT_PATH):
    """True when the reference pack is present."""
    return Path(path).exists()


def reported_maxima(path=DEFAULT_PATH):
    """The absorption and emission maxima the sources state, per dye.

    These differ by a few nm from the peaks of the digitized curves (the Cy3
    curve peaks at 554 nm where the vendors say 550 nm). Figures and listings use
    the reported values so that what is drawn agrees with what is written.
    """
    records = json.loads(Path(path).read_text())
    return {
        key: (float(record["absorption_max_nm"]), float(record["emission_max_nm"]))
        for key, record in records.items()
    }


def load_reference_database(path=DEFAULT_PATH):
    """Build the sample database from the measured reference pack."""
    records = json.loads(Path(path).read_text())
    database = {}
    for key, record in records.items():
        database[key] = Dye(
            name=record["display_name"].split("(")[0].strip(),
            absorption=_band(record, "digitized_absorption", key),
            emission=_band(record, "digitized_emission", key),
            quantum_yield=record["quantum_yield"],
            solvent=record["quantum_yield_solvent"],
            extinction_coefficient=record.get("extinction_coefficient_per_M_per_cm"),
            note=record["notes"],
            source=SOURCE,
        )
    return database
def _band(record, field, key):
    curve = record.get(field)
    if not curve or not curve.get("wavelength_nm"):
        raise ValueError("{} has no {}".format(key, field))
    return TabulatedBand.from_lists(curve["wavelength_nm"], curve["value"], name="{}/{}".format(key, field))
```

File: plsim/detector.py - *Silicon photodiode and its responsivity curve*

```
"""Detection end: silicon photodiode."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np


@dataclass(frozen=True)
class Photodetector:
    """Silicon photodiode converting the selected light into a photocurrent.

    Responsivity R(lambda) is in A/W and the photocurrent is I = R(lambda) * P,
    where P is the optical power reaching the detector. The curve is a smooth
    approximation of a silicon photodiode: weak in the UV, peaking in the near
    infrared, cut off beyond about 1100 nm. Shape model, not a calibration.
    """

    peak_responsivity_aw: float = 0.55
    peak_nm: float = 880.0
    short_sigma_nm: float = 330.0
    long_sigma_nm: float = 110.0

    def responsivity(self, wavelength_nm):
        wl = np.asarray(wavelength_nm, dtype=float)
        sigma = np.where(wl < self.peak_nm, self.short_sigma_nm, self.long_sigma_nm)
        return self.peak_responsivity_aw * np.exp(-0.5 * ((wl - self.peak_nm) / sigma) ** 2)

    def current(self, optical_power, wavelength_nm):
        """Photocurrent for an optical power spectrum, in relative units."""
        return self.responsivity(wavelength_nm) * np.asarray(optical_power, dtype=float)
```

File: plsim/spectrometer.py - *The instrument: one PL scan end to end*

```
"""The full instrument, from lamp to photocurrent."""

from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np

from .detector import Photodetector
from .samples import Dye
from .sources import ExcitationArm, Monochromator


@dataclass(frozen=True)
class PLScan:
    """One simulated PL measurement."""

    sample_name: str
    lambda_exc_nm: float
    wavelengths_nm: np.ndarray
    excitation_power: float
    emitted_power: np.ndarray
    scatter_power: np.ndarray
    emitted_photocurrent: np.ndarray
    scatter_photocurrent: np.ndarray
    photocurrent: np.ndarray
    scatter_fraction: float

    @property
    def detected_power(self):
        """Optical power reaching the detector: PL plus stray excitation light."""
        return self.emitted_power + self.scatter_power

    @property
    def emission_peak_nm(self):
        """Wavelength of the strongest emitted PL."""
        return float(self.wavelengths_nm[int(np.argmax(self.emitted_power))])

    @property
    def scatter_inside_range(self):
        """True when stray excitation light falls inside the detection range."""
        return (
            self.scatter_fraction > 0.0
            and self.wavelengths_nm[0] <= self.lambda_exc_nm <= self.wavelengths_nm[-1]
        )


@dataclass(frozen=True)
class PLSpectrometer:
    """Photoluminescence spectrometer simulator.

    Excitation end: xenon lamp, monochromator set to lambda_exc, harmonic
    filter. Sample: a dye from the sample database. Detection end: monochromator
    scanning lambda_min to lambda_max in steps of dlambda, then a photodiode.
    """
```

```
    sample: Dye
    excitation: ExcitationArm = field(default_factory=ExcitationArm)
    detection_monochromator: Monochromator = field(
        default_factory=lambda: Monochromator(bandwidth_nm=2.0)
    )
    detector: Photodetector = field(default_factory=Photodetector)
    scatter_fraction: float = 0.01
    """Fraction of the excitation light that reaches the detector as stray
    light. The harmonic filter and the detection monochromator reduce it, but
    never remove it."""

    def acquire(
        self,
        lambda_exc_nm=550.0,
        lambda_min_nm=500.0,
        lambda_max_nm=750.0,
        dlambda_nm=1.0,
    ):
        if dlambda_nm <= 0.0:
            raise ValueError("dlambda must be positive")
        if lambda_max_nm <= lambda_min_nm:
            raise ValueError("lambda_max must be greater than lambda_min")

        wavelengths = np.arange(lambda_min_nm, lambda_max_nm + 0.5 * dlambda_nm, dlambda_nm)

        step_nm = float(wavelengths[1] - wavelengths[0]) if wavelengths.size > 1 else dlambda_nm
        detection = self.detection_monochromator

        excitation_power = self.excitation.power_at(lambda_exc_nm)
        emitted_power = detection.throughput * detection.broaden(
            self.sample.emitted_power(wavelengths, lambda_exc_nm, excitation_power), step_nm
        )
        scatter_power = detection.throughput * detection.line_at(
            wavelengths, lambda_exc_nm, self.scatter_fraction * excitation_power
        )

        # Optical powers and photocurrents are kept separate: a figure that draws
        # an optical power on an axis labelled in current is drawn in two units.
        emitted_photocurrent = self.detector.current(emitted_power, wavelengths)
        scatter_photocurrent = self.detector.current(scatter_power, wavelengths)
        photocurrent = emitted_photocurrent + scatter_photocurrent

        return PLScan(
            sample_name=self.sample.name,
            lambda_exc_nm=float(lambda_exc_nm),
            wavelengths_nm=wavelengths,
            excitation_power=excitation_power,
            emitted_power=emitted_power,
            scatter_power=scatter_power,
            emitted_photocurrent=emitted_photocurrent,
            scatter_photocurrent=scatter_photocurrent,
            photocurrent=photocurrent,
            scatter_fraction=self.scatter_fraction,
        )
```

File: plsim/analysis.py - *Excitation scans and their model check*

```
"""Measurements taken with the instrument: excitation spectra and their checks."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from .spectrometer import PLSpectrometer


@dataclass(frozen=True)
class ExcitationScan:
    """PL intensity as a function of excitation wavelength.

    For every excitation wavelength the whole emitted band inside the detection
    range is integrated, so the result depends on how much light the sample
    absorbed, not on a detection window that would go dark for the wrong reason.
    (An earlier version sampled a fixed 565-575 nm window, which collapsed to
    zero once lambda_exc passed 570 nm and produced a fake absorption edge.)

    Every field is either a measurement or the model quantity it is checked
    against, so the scan verifies itself:

    predicted        P_exc * (1 - 10**-A(lambda_exc)) * QY * band_integral
                     -> must reproduce integrated_pl; this checks the sampling,
                     the throughput bookkeeping and linearity.

    Because the emission band shape is independent of the excitation wavelength,
    band_integral is a constant for a fixed detection range, and the measured
    curve must follow the absorbed light exactly.
    """

    wavelengths_nm: np.ndarray
    integrated_pl: np.ndarray
    peak_pl: np.ndarray
    excitation_power: np.ndarray
    absorbed_fraction: np.ndarray
    band_integral: np.ndarray
    quantum_yield: float
    sample_name: str
    detection_range_nm: tuple

    @staticmethod
    def _scaled(curve):
        curve = np.asarray(curve, dtype=float)
        top = curve.max()
        return curve / top if top > 0 else curve

    @property
    def predicted(self):
        """The model's own prediction, including the band integral."""
        return self.excitation_power * self.absorbed_fraction * self.quantum_yield * self.band_integral

    @property
```

```
    def agreement(self):
        """Largest relative difference between measurement and model prediction."""
        measured = self._scaled(self.integrated_pl)
        predicted = self._scaled(self.predicted)
        return float(np.max(np.abs(measured - predicted)))

    @property
    def band_integral_drift(self):
        """How far the band integral moves across the sweep (should be ~0)."""
        band = np.asarray(self.band_integral, dtype=float)
        return float(np.max(np.abs(self._scaled(band) - self._scaled(band[:1]))))

    @property
    def measured_peak_wavelength_nm(self):
        """Excitation wavelength that gives the strongest PL."""
        return float(self.wavelengths_nm[int(np.argmax(self.integrated_pl))])


def excitation_scan(
    spectrometer: PLSpectrometer,
    excitation_wavelengths_nm,
    detection_range_nm=(450.0, 800.0),
    dlambda_nm=1.0,
):
    """Measure the PL intensity while the excitation wavelength is swept."""
    wavelengths = np.asarray(excitation_wavelengths_nm, dtype=float)
    if wavelengths.size < 2:
        raise ValueError("an excitation scan needs at least two excitation wavelengths")

    sample = spectrometer.sample
    throughput = spectrometer.detection_monochromator.throughput

    integrated = np.zeros(wavelengths.size)
    peak = np.zeros(wavelengths.size)
    power = np.zeros(wavelengths.size)
    absorbed = np.zeros(wavelengths.size)
    band = np.zeros(wavelengths.size)

    for index, lambda_exc in enumerate(wavelengths):
        scan = spectrometer.acquire(
            lambda_exc_nm=float(lambda_exc),
            lambda_min_nm=detection_range_nm[0],
            lambda_max_nm=detection_range_nm[1],
            dlambda_nm=dlambda_nm,
        )
        emitted = scan.emitted_power
        if emitted.size:
            integrated[index] = float(np.trapezoid(emitted, scan.wavelengths_nm))
            peak[index] = float(emitted.max())

        grid = scan.wavelengths_nm
        density = sample.emission_shape(grid)
        band[index] = float(np.trapezoid(density, grid)) * throughput

        power[index] = scan.excitation_power
```

```
        absorbed[index] = sample.absorbed_fraction(float(lambda_exc))

    return ExcitationScan(
        wavelengths_nm=wavelengths,
        integrated_pl=integrated,
        peak_pl=peak,
        excitation_power=power,
        absorbed_fraction=absorbed,
        band_integral=band,
        quantum_yield=sample.quantum_yield,
        sample_name=sample.name,
        detection_range_nm=tuple(float(x) for x in detection_range_nm),
    )


@dataclass(frozen=True)
class EmissionBand:
    """Where a simulated emission band sits and how wide it is."""

    peak_nm: float
    fwhm_nm: float


def measured_emission_band(scan):
    """Peak wavelength and FWHM of the PL band in a scan, ignoring stray light."""
    wavelengths = scan.wavelengths_nm
    emitted = scan.emitted_power
    peak_index = int(np.argmax(emitted))
    half = emitted[peak_index] / 2.0
    above_half = wavelengths[emitted >= half]
    return EmissionBand(
        peak_nm=float(wavelengths[peak_index]),
        fwhm_nm=float(above_half[-1] - above_half[0]),
    )
```

File: plsim/plot.py - *Figure rendering*

```
"""Matplotlib rendering of a simulated PL scan."""

from __future__ import annotations

import matplotlib.pyplot as plt
import numpy as np

TRACE_COLOR = "#1f4e9c"
SCATTER_COLOR = "#c1440e"
EXC_COLOR = "#555555"


def plot_scan(scan, path=None, show=False):
    """Draw one PL scan. Returns the figure."""
    power = scan.photocurrent
    scale = float(power.max())
    if scale <= 0.0:
        scale = 1.0

    fig, ax = plt.subplots(figsize=(9.0, 5.6), dpi=140)
    ax.plot(
        scan.wavelengths_nm,
        power / scale,
        color=TRACE_COLOR,
        lw=1.8,
        label="detected PL spectrum",
    )
    if scan.scatter_photocurrent.max() > 1e-3 * scale:
        ax.plot(
            scan.wavelengths_nm,
            scan.scatter_photocurrent / scale,
            color=SCATTER_COLOR,
            lw=1.2,
            ls=":",
            label="stray excitation light",
        )
    ax.axvline(
        scan.lambda_exc_nm,
        color=EXC_COLOR,
        lw=1.0,
        ls="--",
        label="excitation {:g} nm".format(scan.lambda_exc_nm),
    )
    peak_nm = scan.emission_peak_nm
    ax.plot([peak_nm], [1.0], marker="o", ms=4.0, color=TRACE_COLOR)
    ax.annotate(
        "PL peak {:.0f} nm".format(peak_nm),
        xy=(peak_nm, 1.0),
        xytext=(peak_nm + 8.0, 0.92),
        fontsize=9,
        color=TRACE_COLOR,
    )

    ax.set_xlabel("wavelength (nm)")
    ax.set_ylabel("photocurrent (norm.)")
```

```
    ax.set_ylim(0.0, 1.15)
    ax.set_xlim(scan.wavelengths_nm[0], scan.wavelengths_nm[-1])
    ax.grid(alpha=0.25, lw=0.6)
    ax.set_title(
        "{} — PL spectrum, excitation {:g} nm".format(scan.sample_name, scan.lambda_exc_nm)
    )

    settings = (
        "sample: {}\n"
        "excitation: {:g} nm\n"
        "detection range: {:g} - {:g} nm\n"
        "step size: {:g} nm\n"
        "stray excitation: {:g} of excitation power"
    ).format(
        scan.sample_name,
        scan.lambda_exc_nm,
        scan.wavelengths_nm[0],
        scan.wavelengths_nm[-1],
        float(scan.wavelengths_nm[1] - scan.wavelengths_nm[0]),
        scan.scatter_fraction,
    )
    ax.text(
        0.985,
        0.95,
        settings,
        transform=ax.transAxes,
        ha="right",
        va="top",
        fontsize=8,
        family="monospace",
        bbox=dict(boxstyle="round", fc="white", ec="#bbbbbb", alpha=0.9),
    )
    ax.legend(loc="upper left", fontsize=9, framealpha=0.9)
    fig.tight_layout()

    if path is not None:
        fig.savefig(path)
    if show:
        plt.show()
    return fig
```

File: plsim/cli.py - *Command line interface (the user settings)*

```
"""Command line entry point for the PL spectrometer simulator."""

from __future__ import annotations

import argparse
import dataclasses
from pathlib import Path

from .reference import available as reference_available, load_reference_database, reported_maxima
from .samples import SAMPLE_DATABASE, get_sample, with_concentration
from .spectrometer import PLSpectrometer


def build_parser():
    parser = argparse.ArgumentParser(
        prog="plsim",
        description="Photoluminescence spectrometer simulator (xenon lamp, dye samples, photodiode).",
    )
    parser.add_argument("--sample", default="cy3", help="sample from the database (default: %(default)s)")
    parser.add_argument("--database", choices=("auto", "reference", "model"), default="auto",
                        help="reference = measured bands, model = Gaussian bands (default: %(default)s)")
    parser.add_argument("--list-samples", action="store_true", help="print the sample database and exit")
    parser.add_argument("--lambda-exc", type=float, default=550.0, metavar="NM",
                        help="excitation wavelength (default: %(default)s)")
    parser.add_argument("--range", dest="detection_range", type=float, nargs=2, default=(500.0, 750.0),
                        metavar=("LAMBDA_MIN", "LAMBDA_MAX"),
                        help="PL detection range in nm (default: %(default)s)")
    parser.add_argument("--dlambda", type=float, default=1.0, metavar="NM",
                        help="step size for PL detection (default: %(default)s)")
    parser.add_argument("--peak-absorbance", type=float, default=None, metavar="A",
                        help="sample absorbance at the absorption peak (default: 1.0)")
    parser.add_argument("--concentration", type=float, default=None, metavar="MOL_PER_L",
                        help="dye concentration in mol/L; sets the peak absorbance from the extinction coeffic
    ient")
    parser.add_argument("--path-length", type=float, default=1.0, metavar="CM",
                        help="cuvette path length used with --concentration (default: %(default)s)")
    parser.add_argument("--scatter", type=float, default=0.01, metavar="FRACTION",
                        help="fraction of excitation light reaching the detector (default: %(default)s)")
    parser.add_argument("--ideal", action="store_true",
                        help="ideal instrument: no stray excitation light")
    parser.add_argument("--out", type=Path, default=None, metavar="PNG",
                        help="output file (default: out/<sample>_<lambda_exc>nm.png)")
    parser.add_argument("--show", action="store_true", help="open a window instead of writing a file")
    return parser


def choose_database(choice):
    """Return (database, label) for the requested source of sample data."""
    if choice == "model":
        return SAMPLE_DATABASE, "Gaussian model bands"
    if choice == "auto" and not reference_available():
        return SAMPLE_DATABASE, "Gaussian model bands (no reference data found)"
    return load_reference_database(), "measured bands from data/reference_spectra.json"
```

```
def list_samples(database, label, maxima=None):
    """One line per dye. `maxima` gives the values the sources report, which are
    a few nm away from the peaks of the digitized curves."""
    lines = ["sample database: {} ({})".format(label, len(database))]
    for key, dye in database.items():
        if maxima and key in maxima:
            absorption_nm, emission_nm = maxima[key]
        else:
            absorption_nm = _band_peak(dye.absorption)
            emission_nm = _band_peak(dye.emission)
        lines.append(
            "{:<16} {:<16} excitation {:>5.0f} nm  emission {:>5.0f} nm  quantum yield {:g}  ({})".format(
                key, dye.name, absorption_nm, emission_nm, dye.quantum_yield, dye.solvent
            )
        )
    return "\n".join(lines)


def _band_peak(band):
    """Wavelength of the maximum of a band: its centre, or the curve's own peak."""
    if hasattr(band, "center_nm"):
        return band.center_nm
    import numpy as np

    wavelengths = np.asarray(band.wavelength_nm, dtype=float)
    grid = np.arange(wavelengths[0], wavelengths[-1], 0.1)
    return float(grid[int(np.argmax(band(grid)))])


def main(argv=None):
    parser = build_parser()
    args = parser.parse_args(argv)
    try:
        database, label = choose_database(args.database)
    except FileNotFoundError as error:
        parser.error("reference data not found: {}".format(error.filename))

    if args.list_samples:
        maxima = reported_maxima() if reference_available() else None
        print(list_samples(database, label, maxima))
        return 0

    if args.dlambda <= 0.0:
        parser.error("--dlambda must be positive")
    if args.detection_range[1] <= args.detection_range[0]:
        parser.error("--range needs LAMBDA_MIN < LAMBDA_MAX")
    if args.concentration is not None and args.peak_absorbance is not None:
        parser.error("give either --concentration or --peak-absorbance, not both")
    if args.concentration is not None and args.concentration <= 0.0:
        parser.error("--concentration must be positive")

    try:
        sample = get_sample(args.sample, database)
    except KeyError as error:
        parser.error(str(error.args[0]))
```

```
    if args.concentration is not None:
        sample = with_concentration(sample, args.concentration, args.path_length)
    elif args.peak_absorbance is not None:
        sample = dataclasses.replace(sample, peak_absorbance=args.peak_absorbance)

    if not args.show:
        import matplotlib
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    from .plot import plot_scan

    spectrometer = PLSpectrometer(
        sample=sample,
        scatter_fraction=0.0 if args.ideal else args.scatter,
    )
    scan = spectrometer.acquire(
        lambda_exc_nm=args.lambda_exc,
        lambda_min_nm=args.detection_range[0],
        lambda_max_nm=args.detection_range[1],
        dlambda_nm=args.dlambda,
    )

    print("sample           : {} ({})".format(sample.name, sample.solvent))
    print("database         : {}".format(label))
    print("quantum yield    : {:g}".format(sample.quantum_yield))
    print(
        "excitation       : {:g} nm, power {:.4g} (relative units)".format(
            scan.lambda_exc_nm, scan.excitation_power
        )
    )
    print(
        "detection range  : {:g} - {:g} nm, step {:g} nm ({} points)".format(
            scan.wavelengths_nm[0], scan.wavelengths_nm[-1], args.dlambda, scan.wavelengths_nm.size
        )
    )
    print("peak absorbance  : {:g} (peak optical density in the cuvette)".format(sample.peak_absorbance))
    print("absorbed fraction: {:.3f} of the excitation light".format(sample.absorbed_fraction(scan.lambda_exc_
    nm)))
    print("PL peak          : {:.1f} nm".format(scan.emission_peak_nm))
    if scan.excitation_power < 1e-6:
        print("note             : the harmonic filter blocks this excitation wavelength")
    if scan.scatter_inside_range:
        print("note             : stray excitation light shows up at {:g} nm".format(scan.lambda_exc_nm))

    out_path = args.out
    if out_path is None and not args.show:
        slug = sample.name.lower().replace(" ", "-")
        out_path = Path("out") / "{}_{:g}nm.png".format(slug, scan.lambda_exc_nm)
    if out_path is not None:
        out_path.parent.mkdir(parents=True, exist_ok=True)

    figure = plot_scan(scan, path=out_path, show=args.show)
    plt.close(figure)
    if out_path is not None:
        print("figure           : {}".format(out_path))
    return 0
```

File: plsim/__init__.py - *Package interface*

```
"""Photoluminescence spectrometer simulator.

Chain: xenon lamp -> excitation monochromator -> harmonic filter -> sample ->
detection monochromator -> silicon photodiode.
"""

from .analysis import ExcitationScan, excitation_scan, measured_emission_band
from .bands import FWHM_TO_SIGMA, GaussianBand, gaussian
from .detector import Photodetector
from .samples import SAMPLE_DATABASE, Dye, get_sample, with_concentration
from .sources import ExcitationArm, HarmonicFilter, Monochromator, XenonLamp
from .spectrometer import PLScan, PLSpectrometer

__all__ = [
    "Dye",
    "ExcitationArm",
    "ExcitationScan",
    "FWHM_TO_SIGMA",
    "GaussianBand",
    "HarmonicFilter",
    "Monochromator",
    "PLScan",
    "PLSpectrometer",
    "Photodetector",
    "SAMPLE_DATABASE",
    "XenonLamp",
    "excitation_scan",
    "gaussian",
    "get_sample",
    "measured_emission_band",
    "with_concentration",
]
```

File: report/settings.py - *The optical conditions shared by every figure and number*

```
"""
The optical conditions shared by every figure and every number in the report.

They live here, in one place, because they were once written twice: the figures
used 0.1 % stray light while the instrument table in `numbers.md` still said
0.5 %, which would have put a wrong number in the report's own text.
"""

from pathlib import Path

REPO_ROOT = Path(__file__).resolve().parent.parent

CONCENTRATION_M = 1.0e-6
PATH_LENGTH_CM = 1.0
SCATTER_FRACTION = 0.001
DETECTION_RANGE_NM = (450.0, 900.0)

EXCITATION_START_NM = 400.0
EXCITATION_STOP_NM = 700.0
EXCITATION_STEP_NM = 2.0

# Cy3 is transparent at 450 and 650 nm, so those curves would be flat lines at
# zero and would only clutter the question-2 legend.
OVERLAY_EXCITATION_NM = (470.0, 500.0, 520.0, 540.0, 550.0, 560.0)
ASSIGNED_EXCITATION_NM = (450.0, 550.0, 650.0)

DYE_KEYS = ["cy3", "cy5", "methylene-blue"]
DYE_COLORS = {"cy3": "#d95f02", "cy5": "#7570b3", "methylene-blue": "#1b9e77"}
STRAY_COLOR = "#c1440e"

# The digitized absorption curves stop where they were measured. Beyond that the
# model has no data and reads zero absorption, which is a limit of the data, not
# physics: Cy3's absorption is small but non-zero past 590 nm in reality.
ABSORPTION_RANGE_NM = {"cy3": (450.0, 590.0), "cy5": (430.0, 734.0), "methylene-blue": (300.0, 800.0)}
```

File: report/generate_assignment_figures.py - *The figures for questions 1, 2 and 3*

```
"""
Assignment figures for the PL spectrometer simulation.

Question 1: emission spectrum of Cy3, Cy5 and methylene blue.
Question 2: Cy3 with the excitation wavelength swept from 400 nm to 700 nm.
Question 3: the three dyes on one graph at 450 nm, 550 nm and 650 nm excitation.

Fixed settings, which the report must state:
  - all dyes at the SAME CONCENTRATION (1 micromolar, 1 cm cuvette), so the peak
    absorbances differ by their extinction coefficients;
  - detection range wide enough to contain the whole emission band, so a measured
    intensity reflects absorbed light rather than where the window happened to end;
  - stray excitation light at 0.1 % of the excitation power (a double monochromator
    with a harmonic filter rejects most of it; the figure legends say so).

Usage: python report/generate_assignment_figures.py [--database model] [--only part2]
"""

import argparse
import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from plsim.analysis import excitation_scan, measured_emission_band
from plsim.reference import DEFAULT_PATH, load_reference_database
from plsim.samples import SAMPLE_DATABASE, with_concentration
from plsim.spectrometer import PLSpectrometer

sys.path.insert(0, str(Path(__file__).resolve().parent))
from settings import (  # noqa: E402  (same directory, after sys.path)
    ABSORPTION_RANGE_NM,
    ASSIGNED_EXCITATION_NM,
    CONCENTRATION_M,
    DETECTION_RANGE_NM,
    DYE_COLORS,
    DYE_KEYS,
    EXCITATION_START_NM,
    EXCITATION_STEP_NM,
    EXCITATION_STOP_NM,
    OVERLAY_EXCITATION_NM,
    PATH_LENGTH_CM,
    SCATTER_FRACTION,
    STRAY_COLOR,
)

FIG_DIR = Path(__file__).resolve().parent / "report_figures"

plt.style.use("seaborn-v0_8-whitegrid" if "seaborn-v0_8-whitegrid" in plt.style.available else "default")
plt.rcParams.update({
    "font.size": 11,
```

```
    "axes.labelsize": 12,
    "axes.titlesize": 12,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 9,
    "figure.titlesize": 14,
    "figure.dpi": 300,
})


def database(choice):
    source = SAMPLE_DATABASE if choice == "model" else load_reference_database()
    return {key: with_concentration(dye, CONCENTRATION_M, PATH_LENGTH_CM) for key, dye in source.items()}


def absorption_maximum_nm(key):
    """The absorption maximum the sources report (the curve peaks a few nm away)."""
    return float(json.loads(Path(DEFAULT_PATH).read_text())[key]["absorption_max_nm"])


def instrument(dye, scatter=0.0):
    return PLSpectrometer(sample=dye, scatter_fraction=scatter)


def report(rows):
    for row in rows:
        print(row)


def description(dye, lambda_exc, with_name=True):
    """The numbers a reader needs to interpret one curve."""
    return "{}{:.0f} nM in 1 cm, A({:.0f} nm) = {:.3f}, A_peak = {:.3f}, Phi_f = {:g}".format(
        dye.name + ", " if with_name else "", CONCENTRATION_M * 1e9, lambda_exc,
        dye.absorbance(lambda_exc), dye.peak_absorbance, dye.quantum_yield)




# --- instrument diagram -------------------------------------------------


def generate_diagram(dyes):
    """The design itself: the optical train the simulator models."""
    print("Instrument diagram")
    from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

    figure, axis = plt.subplots(figsize=(13.0, 5.6), constrained_layout=True)
    axis.set_xlim(0, 104)
    axis.set_ylim(-2, 36)
    axis.axis("off")

    boxes = [
        ("Xenon arc lamp\n250-1000 nm\nshape model", 8.5),
        ("Excitation\nmonochromator\nlambda_exc, 5 nm", 23.0),
        ("Harmonic filter\npasses > 400 nm", 37.5),
```

```
        ("Sample\ndye in 1 cm cuvette\nA = eps*c*l", 52.5),
        ("Detection\nmonochromator\nlambda_min-lambda_max, dlambda, 2 nm", 69.0),
        ("Silicon photodiode\nR(lambda)\nI = R*P", 88.0),
    ]
    width, height, top = 12.8, 9.4, 25.0

    for label, centre in boxes:
        axis.add_patch(FancyBboxPatch(
            (centre - width / 2, top - height), width, height,
            boxstyle="round,pad=0.35,rounding_size=0.8",
            linewidth=1.2, edgecolor="#2c5282", facecolor="#eef3f9"))
        axis.text(centre, top - height / 2, label, ha="center", va="center", fontsize=8.6, color="#1a2a40")

    for index in range(len(boxes) - 1):
        start = boxes[index][1] + width / 2
        end = boxes[index + 1][1] - width / 2
        axis.add_patch(FancyArrowPatch((start + 0.2, top - height / 2), (end - 0.2, top - height / 2),
                                       arrowstyle="-|>", mutation_scale=13, linewidth=1.3, color="#2c5282"))

    # the user settings act on the two monochromators
    axis.annotate("user setting 1\nexcitation wavelength",
                  xy=(21.5, top - height - 0.4), xytext=(21.5, 9.2), ha="center", fontsize=8.4, color="#c1440e
    ",
                  arrowprops=dict(arrowstyle="->", color="#c1440e", lw=1.1))
    axis.annotate("user setting 2 and 3\ndetection range and step size",
                  xy=(69.5, top - height - 0.4), xytext=(69.5, 9.2), ha="center", fontsize=8.4, color="#c1440e
    ",
                  arrowprops=dict(arrowstyle="->", color="#c1440e", lw=1.1))

    # what the sample does, and the one leak the instrument has to fight
    axis.annotate("PL emitted at lambda > lambda_exc,\nband shape independent of lambda_exc (Kasha)",
                  xy=(52.5, top - height - 0.4), xytext=(46.0, 1.2), ha="center", fontsize=8.4, color="#1b9e77
    ",
                  arrowprops=dict(arrowstyle="->", color="#1b9e77", lw=1.1))
    axis.add_patch(FancyArrowPatch((52.5, top), (69.5, top + 3.4),
                                   arrowstyle="-|>", mutation_scale=12, linewidth=1.1,
                                   color="#c1440e", linestyle="--",
                                   connectionstyle="arc3,rad=-0.28"))
    axis.text(61.5, top + 4.6, "stray excitation light reaching the detector (0.1 % in the figures)",
              ha="center", fontsize=8.4, color="#c1440e")

    axis.text(15.5, top + 1.4, "excitation arm", ha="center", fontsize=9.0, color="#2c5282", style="italic")
    axis.text(69.0, top + 1.4, "detection arm", ha="center", fontsize=9.0, color="#2c5282", style="italic")
    axis.add_patch(FancyArrowPatch((94.5, top - height / 2), (101.5, top - height / 2),
                                   arrowstyle="-|>", mutation_scale=13, linewidth=1.3, color="#2c5282"))
    axis.text(101.5, top - height / 2 - 2.6, "detected\nspectrum", ha="center", fontsize=8.4, color="#4a5568")

    axis.set_title("The instrument the simulator models: every box is a module in the appendix code",
                   fontsize=12.5, fontweight="bold")
    figure.savefig(FIG_DIR / "part1_instrument_diagram.png", dpi=300)
    plt.close(figure)
    return None


# --- question 1 -----------------------------------------------------------
```

```
def generate_part1(dyes):
    print("Part 1: baseline spectra of the three dyes")
    review = []

    for key, dye in dyes.items():
        lambda_exc = absorption_maximum_nm(key)
        scan = instrument(dye, SCATTER_FRACTION).acquire(
            lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
            lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=1.0)
        band = measured_emission_band(scan)
        review.append(
            "  {:<16s} lambda_exc {:5.0f} nm  A(exc) {:.3f}  absorbed {:6.2%}  "
            "emission peak {:6.1f} nm  FWHM {:5.1f} nm  stray/PL {:.3f}".format(
                dye.name, lambda_exc, dye.absorbance(lambda_exc), dye.absorbed_fraction(lambda_exc),
                band.peak_nm, band.fwhm_nm,
                scan.scatter_photocurrent.max() / scan.emitted_photocurrent.max()))

        figure, axis = plt.subplots(figsize=(7.2, 4.6), constrained_layout=True)
        axis.plot(scan.wavelengths_nm, scan.photocurrent, color=DYE_COLORS[key], lw=2.2,
                  label="detected photocurrent (PL + stray)")
        axis.plot(scan.wavelengths_nm, scan.scatter_photocurrent, color=STRAY_COLOR, lw=1.3, ls=":",
                  label="stray excitation light ({:g} % of excitation)".format(SCATTER_FRACTION * 100))
        axis.axvline(scan.lambda_exc_nm, color="gray", ls="--", alpha=0.7,
                     label="excitation {:.0f} nm".format(scan.lambda_exc_nm))
        axis.set_title("Simulated PL spectrum of {}\n{}".format(
            dye.name, description(dye, lambda_exc, with_name=False)), fontsize=11)
        axis.set_xlabel("wavelength (nm)")
        axis.set_ylabel("photodetector current (relative a.u.)")
        axis.legend(loc="upper right")
        figure.savefig(FIG_DIR / "part1_{}.png".format(key), dpi=300)
        plt.close(figure)

    figure, axes = plt.subplots(1, 3, figsize=(16, 4.8), constrained_layout=True)
    for axis, key in zip(axes, DYE_KEYS):
        dye = dyes[key]
        lambda_exc = absorption_maximum_nm(key)
        scan = instrument(dye, SCATTER_FRACTION).acquire(
            lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
            lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=1.0)
        axis.plot(scan.wavelengths_nm, scan.photocurrent, color=DYE_COLORS[key], lw=2.2,
                  label="detected photocurrent")
        axis.plot(scan.wavelengths_nm, scan.scatter_photocurrent, color=STRAY_COLOR, lw=1.2, ls=":",
                  label="stray excitation ({:g} % of excitation)".format(SCATTER_FRACTION * 100))
        axis.axvline(scan.lambda_exc_nm, color="gray", ls="--", alpha=0.7,
                     label="excitation {:.0f} nm".format(scan.lambda_exc_nm))
        axis.set_title("{}\nA({:.0f} nm) = {:.3f}, Phi_f = {:g}".format(
            dye.name, lambda_exc, dye.absorbance(lambda_exc), dye.quantum_yield))
        axis.set_xlabel("wavelength (nm)")
        axis.legend(loc="upper right")
    axes[0].set_ylabel("photodetector current (relative a.u.)")
    figure.suptitle("Part 1: simulated PL spectra, all three dyes at {:.0f} nM in a 1 cm cuvette "
                    "(note: each panel has its own current scale)".format(CONCENTRATION_M * 1e9),
                    fontweight="bold")
```

```
    figure.savefig(FIG_DIR / "part1_combined_overview.png", dpi=300)
    plt.close(figure)
    report(review)


# --- question 2 ----------------------------------------------------------


def generate_part2(dyes):
    print("Part 2: Cy3 with the excitation wavelength swept {:.0f} to {:.0f} nm".format(
        EXCITATION_START_NM, EXCITATION_STOP_NM))
    dye = dyes["cy3"]
    # stray light is on here, so the figure shows the scattered line tracking the
    # excitation wavelength; the sweep on the right uses the PL-only signal, so the
    # stray light cannot contaminate the integrated intensity
    cy3 = instrument(dye, SCATTER_FRACTION)

    figure, (left, right) = plt.subplots(1, 2, figsize=(14.5, 5.6), constrained_layout=True)

    cmap = plt.get_cmap("viridis")
    norm = plt.Normalize(vmin=460.0, vmax=570.0)
    bands = []
    for lambda_exc in OVERLAY_EXCITATION_NM:
        scan = cy3.acquire(lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
                           lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=1.0)
        bands.append(measured_emission_band(scan))
        left.plot(scan.wavelengths_nm, scan.photocurrent, color=cmap(norm(lambda_exc)), lw=2.0,
                  label="excitation {:.0f} nm".format(lambda_exc))
    peaks = [band.peak_nm for band in bands]
    widths = [band.fwhm_nm for band in bands]
    left.set_title("Emission spectra for six excitation wavelengths\n"
                   "peak {:.1f}-{:.1f} nm, FWHM {:.1f}-{:.1f} nm: the band does not move "
                   "(Kasha's rule)".format(min(peaks), max(peaks), min(widths), max(widths)))
    left.set_xlabel("emission wavelength (nm)")
    left.set_ylabel("detected photocurrent (relative a.u.)")
    left.legend(loc="upper right")

    sweep = excitation_scan(cy3, np.arange(EXCITATION_START_NM, EXCITATION_STOP_NM + 0.1,
                                           EXCITATION_STEP_NM))
    measured = sweep.integrated_pl / sweep.integrated_pl.max()
    predicted = sweep.predicted / sweep.predicted.max()
    absorbance = np.array([dye.absorbance(w) for w in sweep.wavelengths_nm])
    absorbance = absorbance / absorbance.max()

    right.plot(sweep.wavelengths_nm, measured, color="#d95f02", lw=2.8,
               label="simulated PL intensity (whole band)")
    right.plot(sweep.wavelengths_nm, predicted, color="#1f4e9c", lw=1.4, ls="--",
               label="model: P_exc x (1 - 10^-A) x Phi_f")
    right.plot(sweep.wavelengths_nm, absorbance, color="#1b9e77", lw=1.4, ls=":",
               label="absorbance A(wavelength), normalized")
    right.set_title("PL intensity while the excitation wavelength is swept\n"
                    "measured vs model vs absorbance")
    right.axvline(ABSORPTION_RANGE_NM["cy3"][1], color="#888888", lw=1.0, ls="-.")
    right.annotate("the measured Cy3 absorption curve ends at {:.0f} nm:\n"
                   "past it the model has no data and reads zero".format(ABSORPTION_RANGE_NM["cy3"][1]),
```

```
                    xy=(ABSORPTION_RANGE_NM["cy3"][1], 0.35), xytext=(ABSORPTION_RANGE_NM["cy3"][1] + 35.0, 0.3
    5),
                    fontsize=8.5, color="#555555",
                    arrowprops=dict(arrowstyle="->", color="#888888", lw=1.0))
    right.set_xlabel("excitation wavelength (nm)")
    right.set_ylabel("normalized intensity")
    right.legend(loc="upper right")

    figure.suptitle("Part 2: excitation wavelength sweep of Cy3 ({:.0f}-{:.0f} nm)\n"
                    "the overlay shows 470-560 nm: Cy3 does not absorb at 450, 600 or 650 nm, "
                    "its measured absorption band stops at {:.0f} nm".format(
                        EXCITATION_START_NM, EXCITATION_STOP_NM, ABSORPTION_RANGE_NM["cy3"][1]),
                    fontweight="bold", fontsize=12.5)
    figure.savefig(FIG_DIR / "part2_cy3_excitation_sweep.png", dpi=300)
    plt.close(figure)

    report([
        "  emission peak across the six wavelengths: {:.1f} to {:.1f} nm".format(min(peaks), max(peaks)),
        "  emission FWHM across the six wavelengths: {:.1f} to {:.1f} nm".format(min(widths), max(widths)),
        "  excitation scan peaks at {:.1f} nm".format(sweep.measured_peak_wavelength_nm),
        "  max |measured - model| = {:.2e} of full scale".format(sweep.agreement),
        "  band integral drift across the sweep = {:.2e}".format(sweep.band_integral_drift),
    ])
    return sweep


# --- question 3 -----------------------------------------------------------


def generate_part3(dyes):
    print("Part 3: three dyes at the same concentration, excited at {}".format(
        ", ".join("{:.0f} nm".format(w) for w in ASSIGNED_EXCITATION_NM)))
    review = []

    # Every curve first, so the panels can be judged against the tallest curve of
    # all of them rather than against the axis of whichever panel is being drawn.
    scans = {}
    for lambda_exc in ASSIGNED_EXCITATION_NM:
        for key in DYE_KEYS:
            dye = dyes[key]
            scans[(lambda_exc, key)] = instrument(dye, 0.0).acquire(
                lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
                lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=1.0)
            review.append(
                "  lambda_exc {:5.0f} nm  {:<16s} A(exc) {:.4f}  absorbed {:6.2%}  peak current {:.5f}".format
    (
                    lambda_exc, dye.name, dye.absorbance(lambda_exc),
                    dye.absorbed_fraction(lambda_exc), scans[(lambda_exc, key)].photocurrent.max()))
    tallest = max(scan.photocurrent.max() for scan in scans.values())

    figure, axes = plt.subplots(1, 3, figsize=(16, 5.2), constrained_layout=True, sharey=True)
    for axis, lambda_exc in zip(axes, ASSIGNED_EXCITATION_NM):
        for key in DYE_KEYS:
            dye = dyes[key]
            scan = scans[(lambda_exc, key)]
```

```
            axis.plot(scan.wavelengths_nm, scan.photocurrent, color=DYE_COLORS[key], lw=2.0,
                      label="{} (A({:.0f} nm) = {:.3f})".format(
                          dye.name, lambda_exc, dye.absorbance(lambda_exc)))
        axis.axvline(lambda_exc, color="black", ls="--", alpha=0.5)
        axis.set_title("excitation {:.0f} nm".format(lambda_exc))
        axis.set_xlabel("wavelength (nm)")
        axis.legend(loc="upper right")
        panel_tallest = max(scans[(lambda_exc, key)].photocurrent.max() for key in DYE_KEYS)
        if panel_tallest < 0.02 * tallest:
            axis.text(0.5, 0.5,
                      "all three dyes are nearly transparent here\n"
                      "absorbed: {:.2f} % (Cy5), {:.2f} % (methylene blue),\n"
                      "negligible for Cy3 - its measured absorption\n"
                      "curve starts at 450 nm\n"
                      "this is why the panel looks empty".format(
                          100 * dyes["cy5"].absorbed_fraction(lambda_exc),
                          100 * dyes["methylene-blue"].absorbed_fraction(lambda_exc)),
                      transform=axis.transAxes, ha="center", va="center", fontsize=8.5,
                      bbox=dict(boxstyle="round", fc="white", ec="#bbbbbb"))
    axes[0].set_ylabel("photodetector current (relative a.u., shared scale)")
    figure.suptitle("Part 3: emission spectra of the three dyes at the same concentration "
                    "({:.0f} nM in 1 cm)\nand the same path length; stray light is left out of "
                    "these panels so the dyes can be compared".format(CONCENTRATION_M * 1e9),
                    fontweight="bold", fontsize=13)
    figure.savefig(FIG_DIR / "part3_three_dyes_comparison.png", dpi=300)
    plt.close(figure)

    # Why the peaks differ: each dye sits at a different point of its own absorption curve.
    figure, axis = plt.subplots(figsize=(10.5, 5.6), constrained_layout=True)
    for key in DYE_KEYS:
        dye = dyes[key]
        grid = np.arange(dye.absorption.wavelength_nm[0], dye.absorption.wavelength_nm[-1], 0.5)
        curve = dye.absorption(grid)
        axis.plot(grid, curve / curve.max(), color=DYE_COLORS[key], lw=2.2,
                  label="{} absorption (peaks at {:.0f} nm)".format(
                      dye.name, grid[int(np.argmax(curve))]))
    for wavelength, style in ((450.0, "-"), (550.0, "--"), (650.0, ":")):
        axis.axvline(wavelength, color="#333333", lw=1.1, ls=style, alpha=0.75)
        axis.text(wavelength, 1.045, "{:.0f} nm".format(wavelength), ha="center", fontsize=9, color="#333333")
    axis.set_title("Why the three peaks differ at the same concentration\nthe three absorption "
                   "curves, with the three excitation wavelengths marked")
    axis.set_xlabel("wavelength (nm)")
    axis.set_ylabel("absorption, normalized to each dye's own maximum")
    axis.set_ylim(0.0, 1.15)
    axis.legend(loc="upper right")
    figure.savefig(FIG_DIR / "part3_absorption_overlay.png", dpi=300)
    plt.close(figure)

    styles = {450.0: (2.4, "-"), 550.0: (1.6, "--"), 650.0: (1.2, ":")}
    figure, axis = plt.subplots(figsize=(11.5, 6.5), constrained_layout=True)
    heights = []
    for lambda_exc in ASSIGNED_EXCITATION_NM:
        for key in DYE_KEYS:
            dye = dyes[key]
            scan = instrument(dye, 0.0).acquire(
```

```
                lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
                lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=1.0)
            width, style = styles[lambda_exc]
            axis.plot(scan.wavelengths_nm, scan.photocurrent, color=DYE_COLORS[key], lw=width,
                      ls=style, label="{} at {:.0f} nm".format(dye.name, lambda_exc))
            heights.append("{} at {:.0f} nm: peak {:.4f}".format(dye.name, lambda_exc, scan.photocurrent.max()
    ))
    axis.set_title("Part 3: all nine spectra on one canvas\nsame concentration, three excitation "
                   "wavelengths; the peak current of every curve is printed on the right")
    axis.set_xlabel("wavelength (nm)")
    axis.set_ylabel("photodetector current (relative a.u.)")
    axis.legend(loc="upper right", fontsize=8.5)
    axis.text(0.985, 0.55, "\n".join(heights), transform=axis.transAxes, ha="right", va="top",
              fontsize=8, family="monospace",
              bbox=dict(boxstyle="round", fc="white", ec="#bbbbbb", alpha=0.9))
    figure.savefig(FIG_DIR / "part3_single_canvas_overlay.png", dpi=300)
    plt.close(figure)
    report(review)


def main(argv=None):
    parser = argparse.ArgumentParser(description="Draw the assignment figures.")
    parser.add_argument("--database", choices=("reference", "model"), default="reference",
                        help="measured reference bands (default) or the Gaussian model")
    parser.add_argument("--only", choices=("all", "part1", "part2", "part3"), default="all",
                        help="draw one question's figures instead of all of them (default: %(default)s)")
    arguments = parser.parse_args(argv)

    FIG_DIR.mkdir(parents=True, exist_ok=True)
    dyes = database(arguments.database)
    print("dye data: {} bands, concentration {:.0f} nM, path length {:g} cm, stray light {:.1%}".format(
        arguments.database, CONCENTRATION_M * 1e9, PATH_LENGTH_CM, SCATTER_FRACTION))

    generate_diagram(dyes)

    actions = {"part1": generate_part1, "part2": generate_part2, "part3": generate_part3}
    chosen = list(actions) if arguments.only == "all" else [arguments.only]
    output = [actions[name](dyes) for name in chosen]
    print("figures written to {}".format(FIG_DIR))
    return output


if __name__ == "__main__":
    main()
```

File: report/generate_numbers.py - *The number table this report quotes from*

```
"""
Write report/numbers.md: every number the report will quote, generated from the
code and the reference pack so that no number is ever retyped by hand.

Run: python report/generate_numbers.py
"""

import argparse
import json
import sys
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from plsim.analysis import excitation_scan, measured_emission_band
from plsim.detector import Photodetector
from plsim.reference import DEFAULT_PATH, load_reference_database
from plsim.samples import SAMPLE_DATABASE, with_concentration
from plsim.spectrometer import PLSpectrometer

sys.path.insert(0, str(Path(__file__).resolve().parent))
from settings import (  # noqa: E402  (same directory, after sys.path)
    ASSIGNED_EXCITATION_NM,
    CONCENTRATION_M,
    DETECTION_RANGE_NM,
    DYE_KEYS,
    PATH_LENGTH_CM,
    SCATTER_FRACTION,
)

OUTPUT = Path(__file__).resolve().parent / "numbers.md"

# Second-opinion verification of 2026-10-18 (see data/SOURCES.md, section
# "Second opinion"): the five numbers the report must not get wrong.
CORRECTIONS = [
    ("methylene blue extinction coefficient", "80000", "67,000 as measured in water "
     "(PhotochemCAD PSW009), 74,028 (OMLC/Prahl), 107,800 for the deconvoluted monomer "
     "(ACS Omega 2020). No source reports 80,000."),
    ("methylene blue emission maximum", "686 nm", "685.0 nm on the shipped digitized curve; "
     "686 nm is used in the published literature on the same band. Say which curve you quote."),
    ("Cy3 emission FWHM", "32 nm", "no published source gives a Cy3 emission bandwidth; it is "
     "measured on the curve shipped here: 28.6 nm (outermost half-maximum crossings) or 30.0 nm "
     "(linear interpolation). Quote it as measured, not cited."),
    ("Cy3 Stokes shift", "20 nm / 638 cm-1", "correct as calculated from the quoted maxima "
     "(550 -> 570 nm = 638.0 cm-1). The digitized curve peaks are 554 and 568 nm, which give "
     "14 nm / 445 cm-1. State which pair you used."),
    ("Cy3 absorbs ~90 % at 550 nm", "90 %", "at the concentration used here (1 uM) Cy3 absorbs "
     "28 %; 90 % needs A = 1.0, i.e. about 6.7 uM. Either state the concentration or drop the "
     "90 % claim."),
]

UNVERIFIED = [
    "primary source for the widely quoted methylene blue extinction coefficient of 95,000 "
```

```
    "(traced no further than repetition; the 1963 paper could not be read from here)",
    "a primary measured value of the methylene blue quantum yield in water (0.02 is the value "
    "used in photodynamic-therapy work; the readable measurement is 0.04 in ethanol)",
    "any published Cy3 or Cy5 emission bandwidth",
    "the provenance of the FPbase dye records (their source field is empty)",
    "whether Cy5 in water is 0.18, 0.20 or 0.28 (0.28 is the central cited value)",
]


def band_fwhm(band, lo, hi):
    """FWHM of a band, on a fine grid, as measured on the shipped curve."""
    grid = np.arange(lo, hi, 0.1)
    values = band(grid)
    half = values.max() / 2.0
    above = grid[values >= half]
    return float(above[-1] - above[0])


def absorption_maximum_nm(key):
    """The absorption maximum the sources report, from the reference pack."""
    return float(records_cache()[key]["absorption_max_nm"])


_RECORDS = {}


def records_cache():
    if not _RECORDS:
        _RECORDS.update(json.loads(Path(DEFAULT_PATH).read_text()))
    return _RECORDS


def band_peak(band, lo, hi):
    grid = np.arange(lo, hi, 0.1)
    return float(grid[int(np.argmax(band(grid)))])


def build(database_choice="reference"):
    records = json.loads(Path(DEFAULT_PATH).read_text())
    source = SAMPLE_DATABASE if database_choice == "model" else load_reference_database()
    dyes = {key: with_concentration(dye, CONCENTRATION_M, PATH_LENGTH_CM) for key, dye in source.items()}

    lines = []
    add = lines.append

    add("# Numbers for the assignment report")
    add("")
    add("Generated by `report/generate_numbers.py` from the code and the reference pack.")
    add("Do not edit by hand: if a number here is wrong, fix the code or the data, not this file.")
    add("")
    add("Assumed optical conditions for every figure and every number below: "
        "**c = 1 umol/L in a 1 cm cuvette** (the same concentration for all three dyes, which is the "
        "premise of question 3), detection range {} nm, model database = `{}`.".format(
            DETECTION_RANGE_NM, database_choice))
    add("")
```

```
add("## 1. Dye database")
add("")
add("| dye | absorption max (source) | emission max (source) | quantum yield (solvent) | "
    "extinction coefficient (1/M/cm) | absorption curve peak | emission curve peak | "
    "emission FWHM (measured) |")
add("| --- | --- | --- | --- | --- | --- | --- | --- |")
peak_wavelengths = {}
for key in DYE_KEYS:
    record = records[key]
    dye = dyes[key]
    absorption_peak = band_peak(dye.absorption, *dye.absorption.wavelength_nm[::len(dye.absorption.wavelen
gth_nm) - 1])
    emission_peak = band_peak(dye.emission, dye.emission.wavelength_nm[0], dye.emission.wavelength_nm[-1])
    width = band_fwhm(dye.emission, dye.emission.wavelength_nm[0], dye.emission.wavelength_nm[-1])
    peak_wavelengths[key] = (record["absorption_max_nm"], record["emission_max_nm"], absorption_peak, emis
sion_peak)
    add("| {} | {} nm | {} nm | {:g} ({}) | {:,} | {:.1f} nm | {:.1f} nm | {:.1f} nm |".format(
        record["display_name"].split("(")[0].strip(),
        record["absorption_max_nm"], record["emission_max_nm"],
        record["quantum_yield"], record["quantum_yield_solvent"],
        record["extinction_coefficient_per_M_per_cm"],
        absorption_peak, emission_peak, width))
add("")
add("Stokes shifts, from the sourced maxima and from the digitized peaks:")
add("")
add("| dye | from sourced maxima | from digitized curve peaks |")
add("| --- | --- | --- |")
for key in DYE_KEYS:
    absorbed, emitted, absorption_peak, emission_peak = peak_wavelengths[key]
    sourced_cm = 1.0e7 / absorbed - 1.0e7 / emitted
    curve_cm = 1.0e7 / absorption_peak - 1.0e7 / emission_peak
    add("| {} | {:.0f} nm / {:.0f} cm-1 | {:.0f} nm / {:.0f} cm-1 |".format(
        key, emitted - absorbed, sourced_cm, emission_peak - absorption_peak, curve_cm))
add("")

add("## 2. The three dyes at the same concentration")
add("")
add("| dye | peak absorbance A | concentration for A = 1 |")
add("| --- | --- | --- |")
for key in DYE_KEYS:
    dye = dyes[key]
    add("| {} | {:.3f} | {:.1f} umol/L |".format(
        dye.name, dye.peak_absorbance, CONCENTRATION_M * 1e6 / dye.peak_absorbance if dye.peak_absorbance
else float("nan")))
add("")
add("Equal concentration does not mean equal absorbance: A_peak = epsilon * c * l, and the "
    "extinction coefficients differ by a factor of {:.1f} between the strongest and the "
    "weakest dye here.".format(
        max(dyes[k].peak_absorbance for k in DYE_KEYS) / min(dyes[k].peak_absorbance for k in DYE_KEYS)))
add("")

add("## 3. The assigned experiment: 450, 550 and 650 nm")
add("")
add("| excitation | dye | A(lambda_exc) | absorbed fraction | emission peak | peak current | integrated PL
```

```
 |")
add("| --- | --- | --- | --- | --- | --- | --- |")
for lambda_exc in ASSIGNED_EXCITATION_NM:
    for key in DYE_KEYS:
        dye = dyes[key]
        scan = PLSpectrometer(sample=dye, scatter_fraction=0.0).acquire(
            lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
            lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=0.5)
        integrated = float(np.trapezoid(scan.emitted_power, scan.wavelengths_nm))
        peak = "{:.1f} nm".format(measured_emission_band(scan).peak_nm) if integrated > 0 else "no emissio
n"
        add("| {:.0f} nm | {} | {:.4f} | {:.2%} | {} | {:.5f} | {:.4g} |".format(
            lambda_exc, dye.name, dye.absorbance(lambda_exc), dye.absorbed_fraction(lambda_exc),
            peak, scan.photocurrent.max(), integrated))
add("")

add("### 3.1 What is drawn below the excitation wavelength, and how much stray light is left")
add("")
add("The emission band is the measured one: the model applies no extra suppression to the "
    "part of the band that lies below lambda_exc, because that part of the spectrum is the "
    "dye's own measured band shape and its amplitude there is small. A real instrument would "
    "record scattered excitation light at the same time, which is the stray-light column.")
add("")
add("| dye | excitation (absorption maximum) | fraction of the emitted band below lambda_exc | "
    "stray light / PL peak |")
add("| --- | --- | --- | --- |")
for key in DYE_KEYS:
    dye = dyes[key]
    lambda_exc = absorption_maximum_nm(key)
    scan = PLSpectrometer(sample=dye, scatter_fraction=0.001).acquire(
        lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
        lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=0.5)
    band = scan.emitted_photocurrent
    below = scan.wavelengths_nm < lambda_exc
    fraction = (
        float(np.trapezoid(band[below], scan.wavelengths_nm[below]) / np.trapezoid(band, scan.wavelengths_
nm))
        if band.max() > 0 else float("nan")
    )
    stray_ratio = (
        scan.scatter_photocurrent.max() / band.max() if band.max() > 0 else float("nan")
    )
    add("| {} | {:.0f} nm | {:.2%} | {:.1%} |".format(
        dye.name, lambda_exc, fraction, stray_ratio))
add("")
grid_fractions = {}
for step in (1.0, 0.5, 0.1):
    values = []
    for key in DYE_KEYS:
        dye = dyes[key]
        scan = PLSpectrometer(sample=dye, scatter_fraction=0.0).acquire(
            lambda_exc_nm=absorption_maximum_nm(key), lambda_min_nm=DETECTION_RANGE_NM[0],
            lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=step)
        band = scan.emitted_photocurrent
        below = scan.wavelengths_nm < absorption_maximum_nm(key)
```

```
        values.append(float(np.trapezoid(band[below], scan.wavelengths_nm[below])
                            / np.trapezoid(band, scan.wavelengths_nm)))
    grid_fractions["{} nm".format(step)] = values
add("That fraction is a numerical integral, so it moves with the integration grid: "
    + "; ".join(
        "{}: {}".format(step, " / ".join("{:.2%}".format(v) for v in values))
        for step, values in grid_fractions.items()
    )
    + " (Cy3 / Cy5 / methylene blue). Quote it as \"a few per cent\", not as a measured number.")
add("")
add("The stray-light figure is a model input, not a measurement: the report figures use 0.1 % "
    "of the excitation power, which is what a double monochromator with a harmonic filter is "
    "expected to leave. With a single monochromator it would be far larger, and methylene blue "
    "at 1 umol/L is dim enough that stray light would then dominate its spectrum.")
add("")
add("Peak wavelengths are read off the scan on the step-size grid after the detection "
    "passband, so they can differ from the tabulated curve peak by about one step "
    "(methylene blue reads 685.5 nm on a 0.5 nm grid and 686 nm on a 1 nm grid; the "
    "curve itself peaks at 685.0 nm). That is the instrument, not an inconsistency.")
add("")
add("The digitized curves are defined only inside the range where they were measured "
    "(Cy3 absorption 450-590 nm, Cy5 430-734 nm, methylene blue 300-800 nm). Outside that "
    "range the model reads zero absorption, which is why Cy3 shows 0.00 % at 450 nm while "
    "the independent check on the same dye puts it near 0.6 %. It does not change the "
    "conclusion at 450 nm, where all three dyes are transparent, but the report should say "
    "\"negligible\" rather than \"exactly zero\".")
add("")
add("## 4. The excitation sweep (question 2)")
add("")
for key in DYE_KEYS:
    instrument = PLSpectrometer(sample=dyes[key], scatter_fraction=0.0)
    sweep = excitation_scan(instrument, np.arange(400.0, 700.1, 2.0),
                            detection_range_nm=DETECTION_RANGE_NM)
    add("- **{}**: strongest PL at {:.1f} nm; measured curve equals "
        "P_exc*(1-10^-A)*Phi_f to {:.1e} of full scale; band integral constant to {:.1e}.".format(
            dyes[key].name, sweep.measured_peak_wavelength_nm, sweep.agreement,
            sweep.band_integral_drift))
add("")
add("Kasha's rule, measured on the simulated band (Cy3, excitation swept inside the absorption "
    "band, the region where a moving spectrum would show up). The width reading depends on the scan "
    "grid: this table uses a 0.5 nm step, the report figure a 1 nm step, which is why the same band "
    "reads 28.5 nm here and 28.0 nm there. The digitized band itself is 28.6 nm wide.")
add("")
add("| excitation | measured emission peak | measured emission FWHM |")
add("| --- | --- | --- |")
instrument = PLSpectrometer(sample=dyes["cy3"], scatter_fraction=0.0)
for lambda_exc in (450.0, 480.0, 510.0, 530.0, 545.0):
    scan = instrument.acquire(lambda_exc_nm=lambda_exc, lambda_min_nm=DETECTION_RANGE_NM[0],
                              lambda_max_nm=DETECTION_RANGE_NM[1], dlambda_nm=0.5)
    if float(np.trapezoid(scan.emitted_power, scan.wavelengths_nm)) <= 0.0:
        add("| {:.0f} nm | no emission | - |".format(lambda_exc))
        continue
    band = measured_emission_band(scan)
    add("| {:.0f} nm | {:.2f} nm | {:.2f} nm |".format(lambda_exc, band.peak_nm, band.fwhm_nm))
add("")
```

```
    add("## 5. Instrument numbers")
    add("")
    lamp = instrument.excitation
    detector = Photodetector()
    add("| quantity | value |")
    add("| --- | --- |")
    add("| excitation monochromator bandwidth | {:.0f} nm |".format(lamp.monochromator.bandwidth_nm))
    add("| detection monochromator bandwidth | {:.0f} nm |".format(instrument.detection_monochromator.bandwidt
    h_nm))
    add("| harmonic filter | passes above {:.0f} nm, blocks below |".format(lamp.harmonic_filter.cutoff_nm))
    add("| stray excitation light | {:.1%} of the excitation power reaches the detector |".format(
        SCATTER_FRACTION))
    for lambda_exc in (450.0, 550.0, 650.0, 700.0):
        add("| relative xenon lamp power at {:.0f} nm | {:.4g} |".format(lambda_exc, lamp.power_at(lambda_exc)
    ))
    for wavelength in (570.0, 670.0, 686.0):
        add("| detector responsivity at {:.0f} nm | {:.3f} A/W |".format(wavelength, detector.responsivity(wav
    elength)))
    add("")

    add("## 6. Numbers the report must not copy from the earlier draft")
    add("")
    add("| claim in the draft | value there | what the sources support |")
    add("| --- | --- | --- |")
    for claim, drafted, supported in CORRECTIONS:
        add("| {} | {} | {} |".format(claim, drafted, supported))
    add("")

    add("## 7. Still unverified")
    add("")
    for item in UNVERIFIED:
        add("- {}".format(item))
    add("")

    return "\n".join(lines) + "\n"


def main(argv=None):
    parser = argparse.ArgumentParser(description="Write report/numbers.md.")
    parser.add_argument("--database", choices=("reference", "model"), default="reference")
    parser.add_argument("--out", type=Path, default=OUTPUT)
    arguments = parser.parse_args(argv)
    text = build(arguments.database)
    arguments.out.write_text(text)
    print("wrote {} ({} lines)".format(arguments.out, text.count("\n")))
    return text


if __name__ == "__main__":
    main()
```

File: tests/test_simulator.py - *The checks behind the physics claims*

```
"""Checks on the physics the report claims, not on the implementation."""

import dataclasses

import numpy as np
import pytest

from plsim.analysis import excitation_scan, measured_emission_band
from plsim.cli import main
from plsim.reference import load_reference_database
from plsim.samples import SAMPLE_DATABASE, get_sample, with_concentration
from plsim.sources import Monochromator
from plsim.spectrometer import PLSpectrometer


def fwhm_nm(x, y):
    """Full width at half maximum of a simple peak."""
    above_half = x[y >= y.max() / 2.0]
    return float(above_half[-1] - above_half[0])


# --- the instrument ----------------------------------------------------------


def test_detection_grid_follows_the_step_size():
    scan = PLSpectrometer(sample=get_sample("cy3")).acquire(
        lambda_exc_nm=520.0, lambda_min_nm=540.0, lambda_max_nm=640.0, dlambda_nm=0.5
    )
    assert scan.wavelengths_nm.size == 201
    assert np.allclose(np.diff(scan.wavelengths_nm), 0.5)


def test_each_dye_emits_at_its_emission_maximum():
    for key, dye in SAMPLE_DATABASE.items():
        scan = PLSpectrometer(sample=dye, scatter_fraction=0.0).acquire(
            lambda_exc_nm=dye.absorption.center_nm,
            lambda_min_nm=dye.absorption.center_nm + 5.0,
            lambda_max_nm=dye.absorption.center_nm + 150.0,
            dlambda_nm=0.5,
        )
        peak = scan.wavelengths_nm[int(np.argmax(scan.emitted_power))]
        assert abs(peak - dye.emission.center_nm) <= 1.0, key


def test_the_readme_dye_table_matches_the_reference_pack():
    """The README drifted from the data once; this keeps it honest."""
    import json
    import re
    from pathlib import Path

    from plsim.reference import DEFAULT_PATH

    records = json.loads(DEFAULT_PATH.read_text())
    readme = (Path(__file__).resolve().parent.parent / "README.md").read_text()
    seen = set()
```

```
    for line in readme.splitlines():
        if not line.startswith("| `"):
            continue
        cells = [cell.strip() for cell in line.strip("|").split("|")]
        if len(cells) != 6 or cells[0].strip("`") not in records:
            continue
        key, _name, absorption, emission, quantum_yield, extinction = cells
        key = key.strip("`")
        seen.add(key)
        record = records[key]
        assert float(re.search(r"[\d.]+", absorption).group()) == record["absorption_max_nm"], key
        assert float(re.search(r"[\d.]+", emission).group()) == record["emission_max_nm"], key
        assert float(re.search(r"[\d.]+", quantum_yield).group()) == record["quantum_yield"], key
        assert float(extinction.replace(",", "")) == record["extinction_coefficient_per_M_per_cm"], key
    assert seen == set(records)


def test_the_monochromator_preserves_the_band_integral():
    """Widening a band must not change how much light is in it."""
    grid = np.arange(450.0, 900.0, 0.5)
    band = get_sample("cy3").emission_shape(grid)
    widened = Monochromator(bandwidth_nm=2.0).broaden(band, 0.5)
    assert np.isclose(np.trapezoid(widened, grid), np.trapezoid(band, grid), rtol=1e-3)


def test_the_monochromator_broadens_the_measured_band():
    dye = get_sample("cy3")
    spectrometer = PLSpectrometer(
        sample=dye,
        scatter_fraction=0.0,
        detection_monochromator=Monochromator(bandwidth_nm=10.0),
    )
    scan = spectrometer.acquire(
        lambda_exc_nm=520.0, lambda_min_nm=530.0, lambda_max_nm=640.0, dlambda_nm=0.25
    )
    measured = fwhm_nm(scan.wavelengths_nm, scan.emitted_power)
    expected = np.hypot(dye.emission.fwhm_nm, 10.0)
    assert abs(measured - expected) < 1.0


def test_excitation_inside_the_range_leaves_a_stray_light_line():
    scan = PLSpectrometer(sample=get_sample("cy5"), scatter_fraction=0.02).acquire(
        lambda_exc_nm=649.0, lambda_min_nm=600.0, lambda_max_nm=750.0, dlambda_nm=0.5
    )
    at_excitation = scan.scatter_power[int(np.argmin(np.abs(scan.wavelengths_nm - 649.0)))]
    assert at_excitation > 10.0 * np.median(scan.scatter_power)


def test_the_harmonic_filter_blocks_ultraviolet_excitation():
    spectrometer = PLSpectrometer(sample=get_sample("cy3"))
    assert spectrometer.excitation.power_at(300.0) < 1e-3 * spectrometer.excitation.power_at(550.0)


# --- the sample ------------------------------------------------------------
```

```
def test_emitted_power_scales_with_quantum_yield():
    dye = get_sample("cy3")
    dim = dataclasses.replace(dye, quantum_yield=dye.quantum_yield / 2.0)
    window = dict(lambda_exc_nm=550.0, lambda_min_nm=560.0, lambda_max_nm=600.0, dlambda_nm=1.0)
    bright_scan = PLSpectrometer(sample=dye, scatter_fraction=0.0).acquire(**window)
    dim_scan = PLSpectrometer(sample=dim, scatter_fraction=0.0).acquire(**window)
    assert np.isclose(bright_scan.emitted_power.max(), 2.0 * dim_scan.emitted_power.max())


def test_cy5_is_the_brighter_dye_at_its_own_optimum():
    cy3 = PLSpectrometer(sample=get_sample("cy3"), scatter_fraction=0.0)
    cy5 = PLSpectrometer(sample=get_sample("cy5"), scatter_fraction=0.0)
    cy3_scan = cy3.acquire(lambda_exc_nm=550.0, lambda_min_nm=560.0, lambda_max_nm=620.0, dlambda_nm=1.0)
    cy5_scan = cy5.acquire(lambda_exc_nm=649.0, lambda_min_nm=655.0, lambda_max_nm=715.0, dlambda_nm=1.0)
    assert cy5_scan.emitted_power.max() > cy3_scan.emitted_power.max()


def test_a_scan_keeps_power_and_current_in_separate_units():
    """A figure that draws an optical power on a current axis is drawn in two units.

    This went wrong once: the stray-light curve was plotted from `scatter_power`
    on an axis labelled in photocurrent, which made it look taller than the
    spectrum it sits on.
    """
    scan = PLSpectrometer(sample=get_sample("methylene-blue"), scatter_fraction=0.001).acquire(
        lambda_exc_nm=664.0, lambda_min_nm=450.0, lambda_max_nm=900.0, dlambda_nm=1.0
    )
    assert np.allclose(scan.photocurrent, scan.emitted_photocurrent + scan.scatter_photocurrent)
    assert np.all(scan.scatter_photocurrent <= scan.photocurrent + 1e-12)
    # in the same units the stray line is a fraction of the PL, never the tallest feature
    assert scan.scatter_photocurrent.max() < scan.emitted_photocurrent.max()


def test_emission_is_linear_in_the_absorbed_light():
    """Doubling what is absorbed doubles the emission, wavelength by wavelength."""
    dye = get_sample("cy3")
    grid = np.array([550.0, 570.0, 590.0])
    once = dye.emitted_power(grid, 550.0, 1.0)
    twice = dye.emitted_power(grid, 550.0, 2.0)
    assert np.allclose(twice, 2.0 * once)


def test_the_emission_band_does_not_move_with_the_excitation_wavelength():
    """Kasha's rule: the spectrum is the same whatever excites the sample."""
    dye = load_reference_database()["cy3"]
    spectrometer = PLSpectrometer(sample=dye, scatter_fraction=0.0)
    bands = [
        measured_emission_band(
            spectrometer.acquire(
                lambda_exc_nm=lambda_exc,
                lambda_min_nm=450.0,
                lambda_max_nm=800.0,
                dlambda_nm=0.5,
            )
```

```
        )
        for lambda_exc in (480.0, 500.0, 520.0, 540.0)
    ]
    peaks = [band.peak_nm for band in bands]
    widths = [band.fwhm_nm for band in bands]
    assert max(peaks) - min(peaks) <= 0.5
    assert max(widths) - min(widths) <= 0.5


def test_there_is_no_cut_of_the_emission_at_the_excitation_wavelength():
    """A step at lambda_exc used to truncate the band; it must not come back."""
    dye = get_sample("cy5")
    grid = np.arange(660.0, 701.0, 1.0)
    emitted = dye.emitted_power(grid, 650.0, 1.0)
    ratio = emitted[1:] / emitted[:-1]
    assert ratio.max() < 1.2


# --- the measured reference data ---------------------------------------------


def test_the_reference_database_peaks_where_the_sources_say():
    import json

    from plsim.reference import DEFAULT_PATH

    records = json.loads(DEFAULT_PATH.read_text())
    database = load_reference_database()
    assert set(database) == set(SAMPLE_DATABASE)
    for key, dye in database.items():
        grid = np.arange(dye.emission.wavelength_nm[0], dye.emission.wavelength_nm[-1], 0.1)
        peak = grid[int(np.argmax(dye.emission(grid)))]
        assert abs(peak - records[key]["emission_max_nm"]) <= 4.0, key


def test_the_reference_pack_is_well_formed():
    """The data file is an input, so its shape and ranges are checked, not assumed."""
    import json

    from plsim.reference import DEFAULT_PATH

    records = json.loads(DEFAULT_PATH.read_text())
    assert set(records) == set(SAMPLE_DATABASE)
    for key, record in records.items():
        for field in (
            "absorption_max_nm",
            "emission_max_nm",
            "quantum_yield",
            "quantum_yield_solvent",
            "extinction_coefficient_per_M_per_cm",
            "digitized_absorption",
            "digitized_emission",
            "notes",
        ):
            assert field in record, (key, field)
```

```
        assert 0.0 < record["quantum_yield"] <= 1.0, key
        assert record["extinction_coefficient_per_M_per_cm"] > 0, key
        for field, quoted_maximum in (
            ("digitized_absorption", record["absorption_max_nm"]),
            ("digitized_emission", record["emission_max_nm"]),
        ):
            wavelengths = np.asarray(record[field]["wavelength_nm"], dtype=float)
            values = np.asarray(record[field]["value"], dtype=float)
            assert wavelengths.size == values.size > 50, (key, field)
            assert np.all(np.diff(wavelengths) > 0), (key, field)
            assert np.all(np.isfinite(values)), (key, field)
            # digitization leaves a few thousandths below zero on the
            # methylene blue absorption curve; anything larger is a real fault
            assert values.min() > -0.01, (key, field, values.min())
            assert abs(values.max() - 1.0) < 1e-6, (key, field)
            assert wavelengths[0] < quoted_maximum < wavelengths[-1], (key, field)


def test_the_reference_database_runs_through_the_instrument():
    dye = load_reference_database()["cy5"]
    scan = PLSpectrometer(sample=dye, scatter_fraction=0.0).acquire(
        lambda_exc_nm=649.0, lambda_min_nm=600.0, lambda_max_nm=780.0, dlambda_nm=0.5
    )
    assert abs(scan.emission_peak_nm - 670.0) <= 4.0


def test_model_and_reference_agree_on_the_band_width():
    measured = load_reference_database()["cy3"]
    x = np.arange(520.0, 640.0, 0.25)
    assert abs(fwhm_nm(x, measured.emission(x)) - get_sample("cy3").emission.fwhm_nm) < 2.0


# --- concentration, which is what the third question turns on --------------


def test_equal_concentration_does_not_mean_equal_peak_absorbance():
    database = load_reference_database()
    one_micromolar = 1.0e-6
    absorbances = {
        key: with_concentration(dye, one_micromolar).peak_absorbance for key, dye in database.items()
    }
    assert np.isclose(absorbances["cy3"], 0.150)
    assert np.isclose(absorbances["cy5"], 0.250)
    assert np.isclose(absorbances["methylene-blue"], 0.067)
    assert absorbances["cy5"] > absorbances["cy3"] > absorbances["methylene-blue"]


def test_concentration_needs_an_extinction_coefficient():
    dye = dataclasses.replace(get_sample("cy3"), extinction_coefficient=None)
    with pytest.raises(ValueError):
        with_concentration(dye, 1.0e-6)


# --- the excitation scan ----------------------------------------------------
```

```
def test_the_excitation_scan_reproduces_the_model():
    """The measured sweep must equal P_exc * (1 - 10**-A) * QY * band integral."""
    for key in ("cy3", "cy5", "methylene-blue"):
        spectrometer = PLSpectrometer(sample=load_reference_database()[key], scatter_fraction=0.0)
        sweep = excitation_scan(
            spectrometer, np.arange(400.0, 701.0, 2.0), detection_range_nm=(450.0, 900.0)
        )
        assert sweep.agreement < 1e-6, key
        assert sweep.band_integral_drift < 1e-9, key


def test_the_excitation_scan_peaks_at_the_absorption_maximum():
    spectrometer = PLSpectrometer(sample=load_reference_database()["cy3"], scatter_fraction=0.0)
    sweep = excitation_scan(spectrometer, np.arange(450.0, 620.0, 1.0))
    assert abs(sweep.measured_peak_wavelength_nm - 550.0) <= 5.0


# --- the command line -----------------------------------------------------


def test_the_cli_rejects_bad_input():
    for arguments in (
        ["--sample", "cy3", "--dlambda", "0"],
        ["--sample", "cy3", "--range", "700", "500"],
        ["--sample", "not-a-dye"],
        ["--sample", "cy3", "--concentration", "1e-6", "--peak-absorbance", "1.0"],
        ["--sample", "cy3", "--concentration", "-1e-6"],
    ):
        with pytest.raises(SystemExit):
            main(arguments)
```